CyberShield: Uncertainty-Aware Fraud Detection with Human-in-the-Loop Calibration for Cybercrime Intelligence

A report submitted for the course named Project - I (CS3201)

Submitted By

ABHISHEK YADAV
SEMESTER - VI
230103041

Supervised By

DR. SALAM MICHAEL SINGH

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY SENAPATI, MANIPUR

APRIL, 2026

Declaration

In this submission, I have expressed my idea in my own words, and I have adequately cited and referenced any ideas or words that were taken from another source. I also declare that I adhere to all principles of academic honesty and integrity and that I have not misrepresented or falsified any ideas, data, facts, or sources in this submission. If any violation of the above is made, I understand that the institute may take disciplinary action. Such a violation may also engender disciplinary action from the sources which were not properly cited or permission not taken when needed.

ABHISHEK YADAV
230103041

Department of Computer Science and Engineering
Indian Institute of Information Technology Senapati, Manipur

Dr. Salam Michael Singh
Assistant Professor

Email: michael_s@iiitmanipur.ac.in
Contact No: +91 8575443081

To Whom It May Concern

This is to certify that the project report entitled **"CyberShield: Uncertainty-Aware Fraud Detection with Human-in-the-Loop Calibration for Cybercrime Intelligence",** submitted to the department of Computer Science and Engineering, Indian Institute of Information Technology Senapati, Manipur in partial fulfillment for the award of degree of Bachelor of Technology in Computer Science and Engineering is a bonafide record of work carried out by **ABHISHEK YADAV** bearing roll number 230103041.

Signature of Supervisor

(Dr. Salam Michael Singh)

Signature of the Examiner 1 .....

Signature of the Examiner 2 .....

Signature of the Examiner 3 .....

Signature of the Examiner 4 .....

Abstract

Cybercrime via SMS, WhatsApp, and URLs represents a rapidly escalating threat necessitating robust automated detection. Conventional machine learning approaches often suffer from poor calibration and high false-positive rates. In this report, we present **CyberShield**, an uncertainty-aware fraud detection framework utilizing a calibrated ensemble model (Logistic Regression, SVM, and Naive Bayes) on high-dimensional TF-IDF vectors. The system employs confidence-bounded thresholding: highly certain malicious patterns ($P(Fraud) \geq 0.90$) trigger automated generation of a pre-filled NCRP-compliant PDF document ready for manual portal submission, while ambiguous cases ($0.30 \leq P(Fraud) < 0.70$) are routed to an administrator panel for Human-in-the-Loop (HITL) annotation. The model is continuously adaptable through a novel *False-Positive-Driven* Hard Negative Mining strategy that selectively mines the top 15% highest-confidence False Positives and retrains with elevated sample weights ($w = 2.0$), improving precision while preserving recall. Extensive evaluation on 26,534 labeled samples demonstrates a test-set accuracy of **94.87%** and an ROC-AUC of **0.9895**. By integrating a Velocity Intelligence tracker for malicious artifacts, CyberShield provides an end-to-end, highly reproducible approach to mitigating sociotechnical cyber-attacks.

Acknowledgement

I would like to express my deepest appreciation to all those who provided me the possibility to complete this project. A special gratitude I give to my supervisor, **Dr. Salam Michael Singh**, whose contribution in stimulating suggestions and encouragement helped me to coordinate my project especially in writing this report.

I would also like to express my sincere gratitude to the Head of the Department (CSE), **Dr. Navanth Saharia**, for his constant support and for providing the necessary facilities to successfully execute this project.

Furthermore, I would also like to acknowledge with much appreciation the crucial role of the faculty and staff of the Department of Computer Science and Engineering, Indian Institute of Information Technology Senapati, Manipur, who gave the permission to use all required equipment and necessary materials to complete the tasks.

Last but not least, many thanks go to my family and friends whose support, guidance, and encouragement have been invaluable throughout this endeavor.

ABHISHEK YADAV
Roll No: 230103041

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Background

In the modern digital era, the proliferation of real-time communication mediums—such as Short Message Service (SMS), WhatsApp, and email—has fundamentally transformed global interconnectedness. While these platforms have brought unprecedented convenience to billions of users, they have simultaneously created an expansive attack surface for malicious actors. Cybercrime, specifically functionally motivated digital fraud, has seen a rapid rise globally. According to the FBI Internet Crime Complaint Center (IC3) 2023 Annual Report [7], billions of dollars are lost annually to sociotechnical cyber-attacks, ranging from phishing, smishing (SMS phishing), and vishing (voice phishing) to advanced fee scams, lottery frauds, and impersonation of government authorities or financial institutions.

Traditional spam filters, built heavily on static blacklists and rudimentary regular expression targeting, have notable limitations. For instance, attackers frequently rotate malicious domains every 48 hours to evade static signature detection, rendering blacklists obsolete almost immediately. Cybercriminals employ sophisticated evasion tactics involving character obfuscation, link shortening services, and highly persuasive semantic phrasing designed to elicit an urgent emotional response from the victim. These attacks exploit human vulnerability rather than system vulnerability. Therefore, identifying these threats requires a deep contextual understanding of natural language, recognizing subtle patterns of urgency, financial requests, and implicit threats that characterize fraudulent communication.

Furthermore, the scale of legitimate communication is so vast that any automated system deployed to filter fraud must maintain a high precision rate to avoid “alert fatigue" and the consequence of suppressing time-critical, legitimate messages (such as legitimate banking alerts or emergency communications). This balance between the false positive rate (FPR) and the true positive rate (TPR) necessitates a paradigm shift from deterministic to probabilistic, uncertainty-aware computational models.

1.2 Motivation

The primary motivation for this project stems from the socioeconomic impact of digital fraud on demographics with lower digital literacy. Attackers frequently utilize local dialects, persuasive psychological triggers, and spoofed authoritative entities (such as banks or government agencies) to bypass the user’s critical thinking. When victims do recognize they have been scammed, the reporting process is often fragmented. In India, for example, the National Cybercrime Reporting Portal (NCRP) provides a centralized mechanism for citizens to report cybercrimes. This is

particularly relevant in regions like Manipur and Northeast India, where digital penetration is accelerating rapidly, yet dedicated local cyber-literacy resources remain scarce, making an automated reporting assistant valuable for the local populace. However, citizens are often unsure *how* to categorize the crime, *what* evidence to preserve, and *where* to find the malicious indicators of compromise (IoCs) within the text payload.

There is a significant gap between the detection of a threat and the actionable intelligence required to report and mitigate it. Standard machine learning models classify text into binary buckets (Fraud vs. Legit). However, the real world operates in a continuum of uncertainty. If an AI system encounters a message it has never seen before, forcing a binary classification often leads to erratic predictions.

This context necessitates an intelligent system capable of not only deciphering fraudulent semantics but also acknowledging its own uncertainty. A system that can confidently flag:

> *"I am 40% sure this is fraud; I require human expert review before deciding."*

represents a safer, calibrated approach to real-world deployment. Consequently, the motivation is to bridge the gap between opaque AI predictions and actionable cyber intelligence by incorporating a Human-in-the-Loop (HITL) calibration and integrating directly with national reporting standards.

1.3 Problem Definition

The core problem addressed in this project is the lack of adaptive, uncertainty-aware, and actionable fraud detection systems for short-text communications. Existing solutions are predominantly binary, lack explainability for non-technical users, and do not provide an end-to-end mechanism for threat reporting.

Specifically, the problem encompasses the following sub-challenges:

1. **High False Positives on Ambiguous Text:** Traditional Natural Language Processing (NLP) classifiers often misclassify borderline legitimate messages that contain financial keywords but lack fraudulent intent.
2. **Concept Drift in Threat Vectors:** Cybercriminals continually alter their phrasing to evade detection models. Static models degrade over time without continuous, curated retraining.
3. **Lack of Explainability:** When an AI flags a message as fraudulent, the user is rarely told *why*, decreasing trust in the system. Without explainability, victims cannot identify which part of a message was suspicious, leaving them vulnerable to identical subsequent attacks.
4. **Fragmented Reporting Workflows:** Once a threat is identified, users lack automated tools to extract the indicators of compromise (IoCs) and format them into an actionable legal complaint.

1.4 Project Objectives

To address the aforementioned challenges, this project, titled **“CyberShield"**, aims to design, develop, and deploy a holistic machine-learning-driven cyber intelligence platform. The main objectives are:

- **Uncertainty-Aware Classification:** To develop a probabilistic machine learning ensemble (combining Logistic Regression, Support Vector Machines, and Naive Bayes) that classifies text across multiple threshold-bounded risk strata (FRAUD, SUSPICIOUS, UNCERTAIN, and LEGIT) rather than forced binary outcomes (See Chapter 4.3).
- **Human-in-the-Loop Calibration:** To implement an administrative intelligence portal where *UNCERTAIN* predictions ($0.30 \leq P(Fraud) < 0.70$) are routed for manual review. This human feedback must trigger a hard-negative mining retraining pipeline to continually calibrate the model against new threats (See Chapter 4.5).
- **Token-Level Explainability:** To engineer a Word Risk Heatmap that visually highlights the specific semantic tokens (words) responsible for the fraud prediction, mapped by their respective coefficient weights in the trained model (See Chapter 5.5).
- **Velocity Intelligence Tracking:** To build a real-time tracking database that extracts and monitors repeating malicious entities (Phone Numbers and URLs) across multiple user submissions, assigning threat levels based on frequency velocity (See Section 4.6.3).
- **Automated NCRP Adjudication:** To provide a one-click automated generation of an NCRP-compliant PDF report, extracting all relevant indicators of compromise (IoCs) and producing a ready-to-submit document for the victim (not a direct API submission to the portal, which is identified as a near-term future enhancement) (See Section 5.5).

1.5 Scope of the Project

The scope of CyberShield is confined to textual analysis of digital communications (SMS, WhatsApp text, Email bodies) primarily in the English language. The system is optimized for English and Romanized Hinglish text. Native script inputs in Devanagari, Meitei Mayek, or other regional scripts are outside the current scope and identified as future work (See Chapter 7.3). The system extracts specific threat vectors, namely suspicious URLs and phone numbers.

The machine learning pipeline is bounded by the current supervised dataset of 26,534 messages. The architecture supports continuous expansion through its dynamic retraining pipeline, which can ingest new HITL-labeled samples in subsequent training cycles without requiring re-engineering. The project does not currently cover deep-packet inspection of network traffic, audio deepfake detection (vishing), or image-based steganography fraud. The user interface assumes deployment as a responsive web application optimized for both desktop and mobile access, facilitating easy copy-pasting of suspicious texts by end-users.

1.6 Project Timeline and Gantt Chart

The development of CyberShield was structured iteratively over several months, adopting an iterative, sprint-based development approach. The timeline below outlines the distinct phases from requirement gathering to final evaluation and documentation spanning from late January to mid-April 2026.

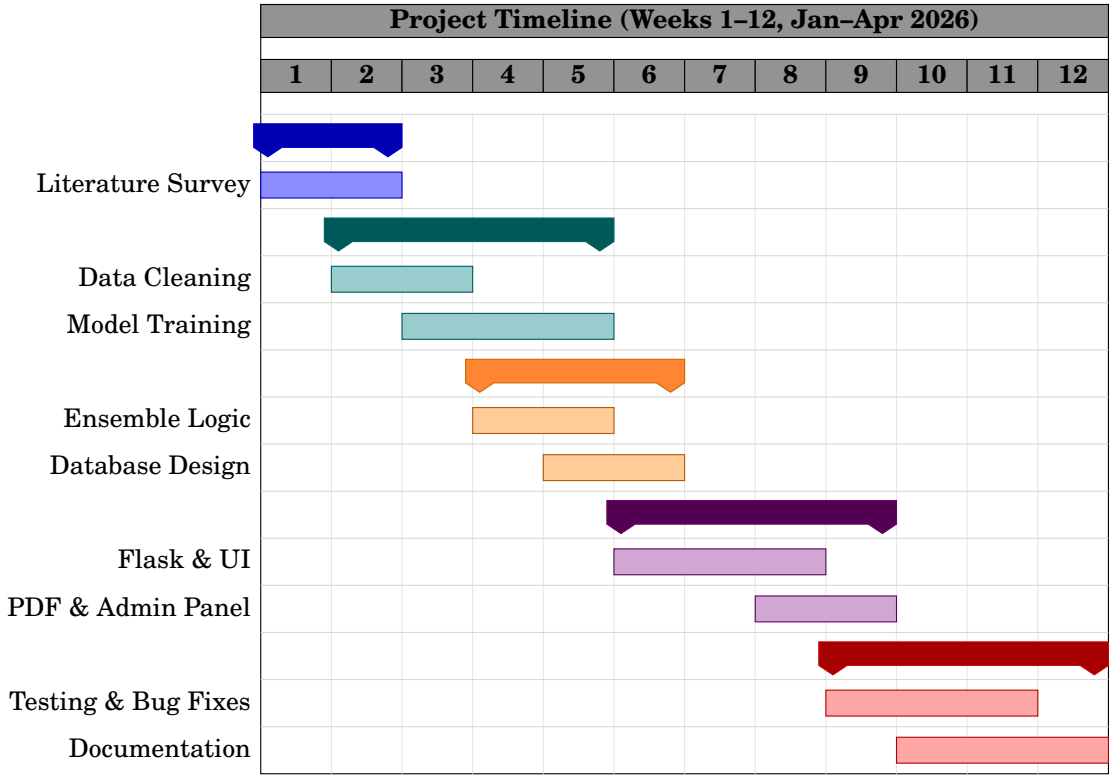

Figure 1.1: CyberShield Project Development Timeline (Jan–Apr 2026) across 12 weekly sprints. Each phase is color-coded: Research (blue), ML Pipeline (teal), Architecture (orange), Web Application (purple), Evaluation (red).

The phased approach ensured that the core machine learning components were validated before integration into the Flask web application. Each phase concluded with a checkpoint review confirming that probability arrays passed correctly between the offline training pipeline and the online inference server.

Chapter 2

Literature Survey

2.1 Overview

The rapid proliferation of digital communication channels has been matched by an equally rapid evolution in cybercrime methodologies. Traditional approaches to identifying unsolicited, malicious, or fraudulent electronic messages have transitioned from rudimentary rule-based filtering to sophisticated, high-dimensional machine learning frameworks. This chapter reviews the state-of-the-art literature relevant to the foundational pillars of the CyberShield project: Short Message Service (SMS) and text-based fraud detection, threshold uncertainty and active learning, and the integration of Human-in-the-Loop (HITL) methodologies in cybersecurity. This chapter is organized into four sections: classical ML approaches to spam detection (Section 2.2), NLP tooling (Section 2.3), Human-in-the-Loop and active learning paradigms (Section 2.4), and a gap analysis situating CyberShield within the existing literature (Section 2.5).

2.2 Machine Learning in Spam and Fraud Detection

The academic investigation into automated filtering of electronic messages conceptually began with email spam. However, due to the constrained character limits, informal syntactic structures, and specific colloquialisms prevalent in SMS and instant messaging platforms (like WhatsApp), traditional email detection algorithms perform poorly when directly transposed to short-text environments.

2.2.1 Evolution of Text Vectorization and Modeling

Almeida et al. [2] introduced the SMS Spam Collection v.1, which remains one of the most widely used public benchmarks for SMS spam classification research. Their comparative study of standard algorithms - including Support Vector Machines (SVM) and Naive Bayes (NB) - established that Term Frequency–Inverse Document Frequency (TF-IDF) vectorization combined with linear classifiers produces competitive baseline metrics on short-text data. Subsequent work has demonstrated that adversarial obfuscation techniques (character substitution, homoglyph attacks, whitespace manipulation) can degrade detector performance, motivating the need for character-level features and dynamic retraining pipelines such as those employed in CyberShield.

Building upon this, Abid et al. [1] conducted a comprehensive comparative analysis of traditional machine learning against deep learning methodologies for SMS spam detection. Their research demonstrated that while Deep Learning models (like LSTMs and CNNs) achieve modest performance gains on highly complex, non-linear linguistic structures, traditional algorithms

(specifically optimized SVM and Logistic Regression models) provide significantly faster inference times and lower computational overhead, making them well-suited for real-time deployment. Furthermore, tree-based and linear models provide inherent feature explainability, which is a critical requirement in forensic cybercrime investigations.

Similarly, Roy et al. [17] investigated deep learning mechanisms but emphasized the importance of extensive dataset curation. Their findings underscored the “garbage in, garbage out" paradigm; highly imbalanced datasets result in models that collapse towards the majority class (typically legitimate messages). Consequently, optimal detection relies heavily on synthetic minority oversampling (SMOTE) or hard negative mining to penalize the misclassification of edge-case fraud vectors.

2.2.2 Smishing and Phishing URL Detection

A significant subset of text-based cybercrime relies on delivering malicious payloads via Uniform Resource Locators (URLs), broadly categorized as Smishing (SMS Phishing). Jain and Gupta [13] proposed an integrated machine learning approach that does not solely rely on the semantic text of the message, but actively parses the embedded URLs to extract lexical features (e.g., URL length, presence of hyphens, highly entropic subdomains). Their work demonstrated that combining NLP semantic analysis with lexical URL analysis reduces the false positive rate.

Further solidifying this, Sahingoz et al. [18] demonstrated that lexical URL features alone - domain length, character entropy, presence of suspicious tokens - can achieve over 97% phishing detection accuracy on large-scale datasets. The consensus across multiple reviewed studies indicates that attackers frequently exploit URL shortening services (e.g., bit.ly, tinyurl) to obfuscate malicious domains from localized text scanners. To counter this, CyberShield incorporates a Velocity Intelligence module specifically designed to track the submission frequency of these shortened artifacts, flagging them as malicious based on submission volume regardless of the domain’s reputation at the time of submission.

2.3 Natural Language Processing Tooling

The implementation of machine learning models requires robust frameworks capable of transforming raw, unstructured text into mathematical representations. The Natural Language Toolkit (NLTK) established the academic standard for programmatic linguistics. Their comprehensive documentation covers the necessity of stop-word removal, stemming, and lemmatization to reduce the dimensionality of the vector space, ensuring that algorithms are not distracted by syntactically necessary but semantically hollow conjugations.

Although contextual embedding approaches such as Word2Vec and BERT offer richer semantic representations, two practical constraints favored TF-IDF for this deployment: (i) the inference latency budget of 500ms per request, which contextual transformer models exceed on CPU-only infrastructure, and (ii) the requirement for token-level explainability via inspectable model coefficients, which is straightforward for linear models over TF-IDF features but opaque for transformer embeddings. For the predictive modeling phase, the *scikit-learn* library has become the de facto standard in the machine learning community. Their architecture provides standardized, interchangeable modules for vectorization, enabling researchers to rapidly prototype ensemble mechanisms. CyberShield utilizes these exact theoretical mathematical constructs to perform its soft-voting probability aggregation.

2.4 Human-in-the-Loop (HITL) and Active Learning

A glaring deficiency in modern commercial fraud filters is their deterministic binary nature. Models are mathematically forced to classify a message as 1 (Fraud) or 0 (Legit), often making arbitrary decisions on messages that sit squarely on the algorithmic decision boundary.

2.4.1 The Necessity of the Human Operator

Holzinger [12] explored this paradox extensively in the context of health informatics, positing the question: *When do we need the human-in-the-loop?* Holzinger argues that in domains where the cost of a false positive is catastrophic, fully autonomous “black-box" AI is suboptimal. Instead, algorithms should be utilized to perform the heavy lifting of parsing vast datasets, extracting the ambiguous “uncertain" samples, and presenting them to a human expert. The human leverages contextual domain knowledge that the AI lacks, annotates the ambiguous sample, and feeds it back into the training loop. This specific philosophy explicitly underpins CyberShield's multi-tier thresholding logic, where probabilities between 0.30 and 0.70 are deferred to admin authorities rather than guessed autonomously.

2.4.2 Concept Drift and Active Learning Paradigms

This mechanism of querying a human expert is formally known in academic literature as Active Learning. Settles [20] provides the definitive survey on Active Learning, outlining various query strategies. The most relevant to threat intelligence is “Uncertainty Sampling," where the model queries the human operator for the labels of instances about which it is least certain. Furthermore, active learning directly mitigates the phenomenon of *Concept Drift* [9], where the statistical properties of the target variable (e.g., spam patterns) change rapidly over time. Settles shows that selecting boundary cases for human labelling enables a machine learning model to reach a target accuracy with substantially fewer labelled instances than random passive sampling.

2.4.3 Hard Negative Mining

Addressing class imbalance and boundary ambiguity programmatically leads to the concept of Hard Negative Mining. Shrivastava, Gupta, and Girshick [21] presented an online hard example mining algorithm initially designed for region-based object detectors in computer vision. While originating in computer vision, the core principle of heavily penalizing high-confidence errors transfers directly across domains and is highly effective when adapted for NLP fraud detection. In the context of CyberShield, a “hard negative" is a highly suspicious legitimate message (e.g., an urgent banking OTP) that the model incorrectly flags as a scam. By programmatically extracting these failures during the cross-validation phase and forcing the model to retrain explicitly on them, the decision boundary is shifted away from the legitimate class distribution, reducing the risk of false alarms on borderline cases.

2.5 Gap Analysis and CyberShield Contribution

The reviewed literature reveals a significant disconnect between theoretical machine learning accuracy and practical, real-world deployment in cybercrime mitigation. Most studies conclude upon achieving a marginally superior accuracy metric on static, sterile datasets.

There is a notable gap in literature regarding end-to-end socio-technical pipelines. Academic models do not translate their raw probabilistic outputs into actionable intelligence for the victim (such as an automated NCRP FIR schema). Furthermore, public datasets suffer from extreme temporal decay; a model trained predominantly on 2011-era SMS spam vectors (such as the Almeida dataset [2]) is likely to underperform against contemporary WhatsApp KYC and PAN-card update scams that did not exist when the dataset was compiled.

CyberShield addresses this gap by combining the established robustness of TF-IDF ensemble modelling with Holzinger's Human-in-the-Loop paradigm [12] and iterative Hard Negative Mining [21]. The result is an auto-calibrating fraud detection platform that integrates with national cybercrime reporting standards rather than producing only research-grade metrics.

2.5.1 Comparison with Recent Systems

To definitively contextualize the novelty of CyberShield, Table 2.1 illustrates a direct architectural comparison between recent notable academic systems and our proposed architecture.

System / Authors	Algorithm Core	Artifact Neutralization	HITL Calibration	Uncertainty Handling	Primary Limitation / Gap
Almeida et al. (2011) [2]	SVM, Naive Bayes	No	No	None	Binary only; high false positive rate on edge cases.
Roy et al. (2020) [17]	CNN / LSTM	No	No	None	Computationally heavy; lacks explainability for cyber intelligence.
Jain & Gupta (2021) [13]	Lexical URL + NLP	Yes (URLs only)	No	None	Does not extract Phone Numbers; no feedback loop for concept drift.
Abid et al. (2022) [1]	Optimized LR / SVM	No	No	None	Black-box predictions without automated legal reporting integration.
CyberShield (Current)	**Ensemble (LR+SVM+NB)**	**Yes (URLs + Phones)**	**Yes (Active Learning)**	**4-tier Probabilistic**	**Currently English/Hinglish only; no native multilingual support.**

Table 2.1: Architectural Comparison of CyberShield against contemporary spam detection systems.

Chapter 3

Requirement Engineering

3.1 Introduction

Requirement Engineering is the foundational phase of the software development lifecycle, encompassing the systematic process of gathering, analyzing, documenting, and validating the specifications of the proposed system. For the CyberShield portal, requirements are bifurcated into requirements dealing directly with the machine learning inference pipeline (backend intelligence) and those concerning the interaction logic of the front-end user interfaces (both citizen-facing and admin-facing). Functional requirements define what the system must do; non-functional requirements define how well it must do it.

3.2 System Analysis

3.2.1 Limitations of the Existing System

Before designing CyberShield, a thorough analysis of existing generic spam filters and cybercrime complaint mechanisms was conducted. The primary limitations identified were:

1. **Binary Rigidity:** Existing text classifiers typically enforce a hard binary output. Messages that are highly ambiguous are frequently misclassified, offering no gradient of risk or uncertainty.
2. **Opaque Decision Making:** Currently, when an email provider routes a message to the "Spam" folder, the user is not informed *why*. This black-box approach breeds user distrust, particularly when false positives occur.
3. **Manual Intelligence Gathering:** Victims of cybercrime must manually copy phone numbers, isolate URLs, and draft complaints from scratch when filing a First Information Report (FIR) on the NCRP portal, a process prone to transcription errors and incomplete artifact capture.
4. **Static Models:** Most commercial off-the-shelf classifiers employ models that decay rapidly. Retraining these models requires extensive data engineering rather than a streamlined Human-in-the-Loop pipeline. Furthermore, retraining cycles in commercial systems are typically quarterly or annual, whereas cybercrime phrasing evolves on a daily basis.

3.2.2 Feasibility Study

The feasibility of developing CyberShield was evaluated across three dimensions:

- **Technical Feasibility:** The implementation of TF-IDF vectorization and classical ensemble modeling (SVM/LR/NB) on a corpus of 26,500 messages is technically feasible with low computational overhead. The system can easily be hosted on standard cloud infrastructure (e.g., Render, Heroku) using a Python/Flask ecosystem. Thus, the project is technically highly feasible.
- **Operational Feasibility:** By designing an intuitive citizen-facing interface and an OAuth 2.0 secured Admin panel for HITL adjudication, the system aligns perfectly with the operational workflows of civilian users and cyber-intelligence officers.
- **Economic Feasibility:** As the project relies entirely on open-source libraries (scikit-learn, ReportLab, Flask) and lightweight SQLite databases, capital expenditure is minimized to server hosting costs alone. Estimated monthly hosting cost on platforms such as Render or Railway is approximately USD 0–7, making the system highly cost-effective for institutional deployment.

3.3 System Requirements

3.3.1 Hardware Requirements

For deployment and continuous active retraining (Hard Negative Mining), the server infrastructure requires specific minimum capabilities to ensure the 25,000-feature TF-IDF matrix can be processed within RAM without triggering swap space bottlenecks.

- **Minimum Client Device (End User):**
  - Any device with a modern browser (Chrome 90+, Firefox 88+, Safari 14+).
  - Minimum 1Mbps internet connection.
  - No local computation required — all inference is server-side.
- **Development Environment:**
  - **Processor:** Minimum Intel Core i5 or AMD Ryzen 5 (multi-threading heavily utilized during cross-validation).
  - **RAM:** 8 GB minimum; 16 GB recommended for manipulating the 100MB+ master dataset arrays during vectorization.
  - **Storage:** 256 GB SSD for fast I/O read/write operations during model pickling operations.
- **Production Server (Minimum):**
  - **CPU:** 2 virtual Cores (vCPU).
  - **RAM:** 1 GB (Model inferences are lightweight, taking $< 100ms$).
  - **Storage:** 20 GB block storage.

3.3.2 Software Requirements

The software stack is uniformly Pythonic, minimizing impedance mismatch between the data science layer and the web server layer.

- **Operating System:** Agnostic (Windows 11 during development; Linux/Ubuntu for deployment).
- **Programming Language:** Python 3.10+ is required specifically for structural pattern matching and improved dictionary ordering used in the threshold routing logic.
- **Machine Learning Layer:**
  - `scikit-learn`: Core algorithm implementations (SVM, Logistic Regression, Naive Bayes, TF-IDF).
  - `pandas / numpy`: Vectorized data frame manipulation and array operations (used exclusively in the offline `pipefinal.py` training pipeline, not in the runtime web server).
  - `joblib`: Fast disk-caching and efficient binary serialization of trained model objects.
- **Web Framework & Backend:**
  - `Flask`: Micro-framework handling HTTP routing and Jinja2 templating.
  - `flask-dance`: Implementation of secure Google OAuth 2.0 authentication.
- **Database & Session Layer:**
  - `Flask-SQLAlchemy` & `psycopg2-binary`: ORM mapping and thread-safe database connection pooling.
  - `Flask-Migrate`: Handling database schema migrations.
  - `Flask-Login`: User session management.
- **Artifact Generation:**
  - `ReportLab`: Programmatic generation of the NCRP-compliant PDF utilizing Canvas draw operations.

3.4 Functional Requirements

Functional requirements define the core behavior, inputs, outputs, and specific computational tasks the system must perform.

1. **Message Ingestion & Parsing:** The system must permit users to paste text up to 800 characters (enforced at the HTML frontend layer via the `maxlength` attribute, not the server-side backend) and systematically strip whitespace and non-printable characters.
2. **Artifact Extraction:** The system must execute RegEx pipelines to extract and isolate all URLs (http/https/www) and 10+ digit numerical vectors resembling phone numbers.
3. **Probabilistic Scoring:** The system must compute a soft-voting probability matrix against the text string using ensemble weights determined via grid search over the validation set, with the empirically optimal values for the current training run being (SVM=0.50, LR=0.20, NB=0.30) and classify it into bounds:

- FRAUD: $P(\text{Fraud}) \geq 0.90$
- SUSPICIOUS: $0.70 \leq P(\text{Fraud}) < 0.90$
- UNCERTAIN: $0.30 \leq P(\text{Fraud}) < 0.70$
- LEGIT: $P(\text{Fraud}) < 0.30$

4. **Word Risk Heatmap:** The system must map coefficients from the underlying Logistic Regression model back to the input string tokens, coloring them by severity to explain the prediction.
5. **Velocity Intelligence:** The system must update a global datastore incrementing the occurrence count of extracted phone numbers/URLs, flagging them as MED/HIGH/CRITICAL based on frequency.
6. **NCRP Document Generation:** The system must dynamically render a PDF embedding the extracted artifacts, risk level, raw text, and a generated NCRP Complaint Category Code.
7. **HITL Administrative Annotation:** The system must securely present *UNCERTAIN* anomalies to an OAuth-authenticated admin for manual binary labeling. The admin interface must display the raw message, extracted artifacts, and the model's probability score alongside the labeling controls to provide sufficient context for accurate human adjudication.
8. **Automated Subprocess Retraining:** The admin panel must possess a trigger to launch an asynchronous subprocess that merges new HITL labels, extracts hard negatives, and retrains the entire mathematical model from scratch, subsequently overwriting the pickled artifacts. The retraining subprocess must preserve the frozen test set indices from `split_indices.pkl` to prevent contamination of the evaluation partition. After retraining completes, the admin can reload model weights into RAM via the `/admin/reload-model` route without restarting the Flask server.
9. **Colloquial Override:** The system must suppress false positives on standard conversational phrases by applying a deterministic override that forces $P(\text{Fraud}) = 0.05$ when the input matches a predefined conversational dictionary and the raw ensemble probability is below 0.50.
10. **Authentication Requirement:** Google OAuth authentication is required before a user can submit a message for analysis.
11. **User Dashboard:** Registered users must be able to access a complaint history dashboard.
12. **Role Management:** Admins must be able to manage user roles (promote/demote) via a dedicated user management panel.

3.5 Non-Functional Requirements

Non-functional requirements dictate the operational constraints, performance, scaling, and holistic attributes of the system.

1. **Inference Latency:** Real-time analysis of a 500-character string (including vectorization, predict_proba calls, and regex artifact extraction) must not exceed 500 milliseconds under normal load. This threshold aligns with established UX research indicating that response times under 500ms are perceived as immediate by users, preventing abandonment of the reporting interface.

2. **Security & Authentication:** Administrative routes (retraining, velocity DB access, HITL labeling) must be strictly gated behind Google OAuth 2.0, utilizing secure, HttpOnly session cookies.

3. **Idempotency of the Test Set:** The system's retraining module must strictly utilize a frozen `split_indices.pkl` list to guarantee that the 3,980 evaluation instances remain completely uncontaminated by new HITL data, ensuring consistent scientific reproducibility of the ROC-AUC and F1 metrics.

4. **Aesthetics and UX:** The citizen-facing frontend must utilize responsive, accessible, dark-themed (glassmorphic) interfaces that do not intimidate victims of cybercrime.

5. **Concurrent Access:** The system must handle at least 10 simultaneous citizen inference requests without degradation in response time, achieved through SQLAlchemy's connection pool for thread-safe database operations, replacing previous raw SQLite3 WAL approaches.

Chapter 4

System Design

4.1 Introduction to System Design

System Design translates the abstract logical requirements into a concrete blueprint detailing how the software components will interact, how data will flow through the mathematical pipelines, and how the user interfaces will trigger underlying backend inferences. For CyberShield, the architecture is designed as a modular intelligence platform, encapsulating data processing, machine learning inference, Human-in-the-Loop (HITL) queuing, and PDF generation within a unified operational context.

4.2 System Architecture Concept

The overall architecture follows a Model-View-Controller (MVC) paradigm, distinctively augmented by an offline Machine Learning (ML) pipeline loop. The backend operates as a modular monolith. The ML pickle files act as a completely separate service layer distinct from the traditional database Model layer, ensuring that model inference logic remains decoupled from standard data retrieval.

- **The View (Presentation Layer):** Hosted via Flask templates, this layer provides two drastically different interfaces based on OAuth scopes. The citizen view focuses on minimalistic text input and PDF output. The admin view provides rich dashboard metrics, pending label queues, and retraining controls.
- **The Controller (Routing & Logic):** The Flask `app.py` file dictates business logic. It handles the RESTful routes, manages active user sessions via secure cookies, processes Form Data, and invokes the computational sub-modules (prediction, velocity logging, PDF generation).
- **The Model (Intelligence Layer):** Contains two facets: the SQLite database (`cybershield.db`) which holds stateless records of complaints and artifacts, and the Pickled Serialization space (`final_vectorizer.pkl`, `final_models.pkl`) which loads the active RAM state of the mathematical ensemble.
- **The Retrain Pipeline (Asynchronous Daemon):** An isolated subsystem (`pipefinal.py`) that operates entirely offline. It connects to the database, extracts new truths, recalculates TF-IDF boundaries, identifies hard negatives, cross-validates, and writes new models to disk without blocking the main Web Server.

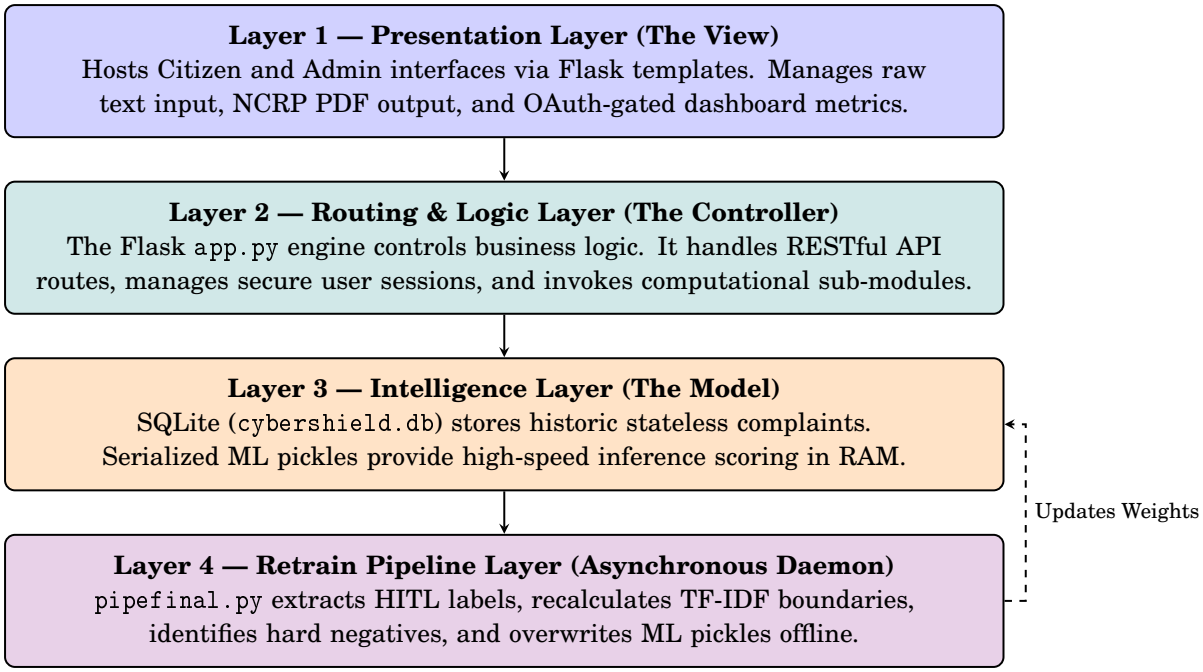

Figure 4.1: CyberShield Modular System Architecture — restructured into a four-layer processing pipeline. Adjacent layers communicate sequentially, while the asynchronous Retrain Daemon operates independently to push updated mathematical weights back into the Intelligence Layer.

4.3 Use Case Analysis

To map the functional requirements into actionable system workflows, a comprehensive Use Case analysis was conducted. Figure 4.2 illustrates the interaction between the primary actors (Citizen and Administrator), the secondary authentication actor (Google OAuth Server), and the encapsulated processes within the CyberShield system boundary.

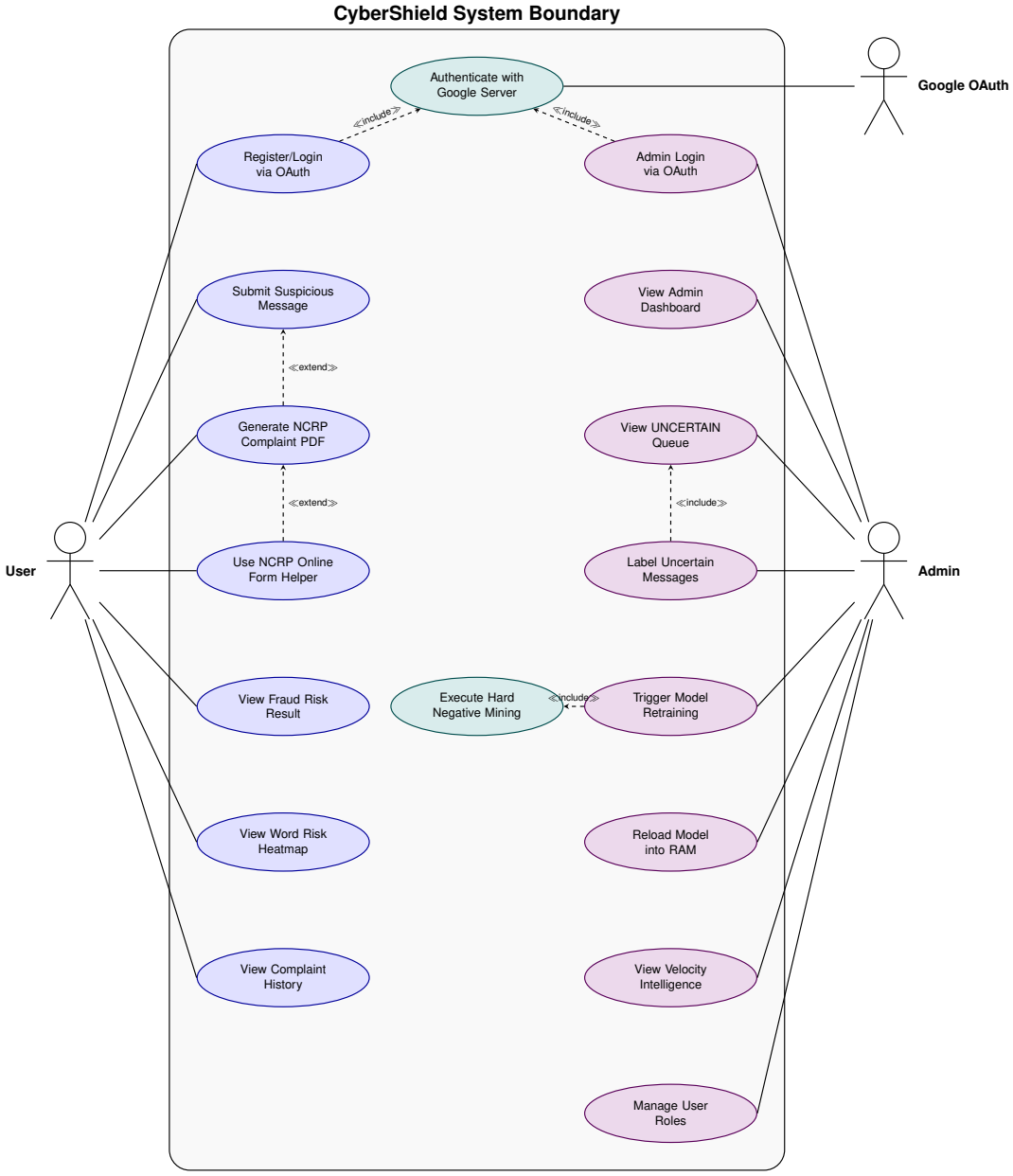

Figure 4.2: Formal UML Use Case Diagram for CyberShield — demonstrating the functional partition between public Citizen workflows and privileged Administrator controls, unified by a central Google OAuth authentication layer.

4.4 Data Flow Diagrams

A Data Flow Diagram (DFD) is a structured graphical tool that models the flow of information through a system, its transformations, storage, and interactions with external entities. Two levels are presented here.

4.4.1 Level 0 DFD — Context Diagram

The Level 0 DFD, also known as the Context Diagram, represents the entire CyberShield system as a single process (Process 0) and defines its boundaries by identifying all external entities that interact with it, along with the high-level data flows between them.

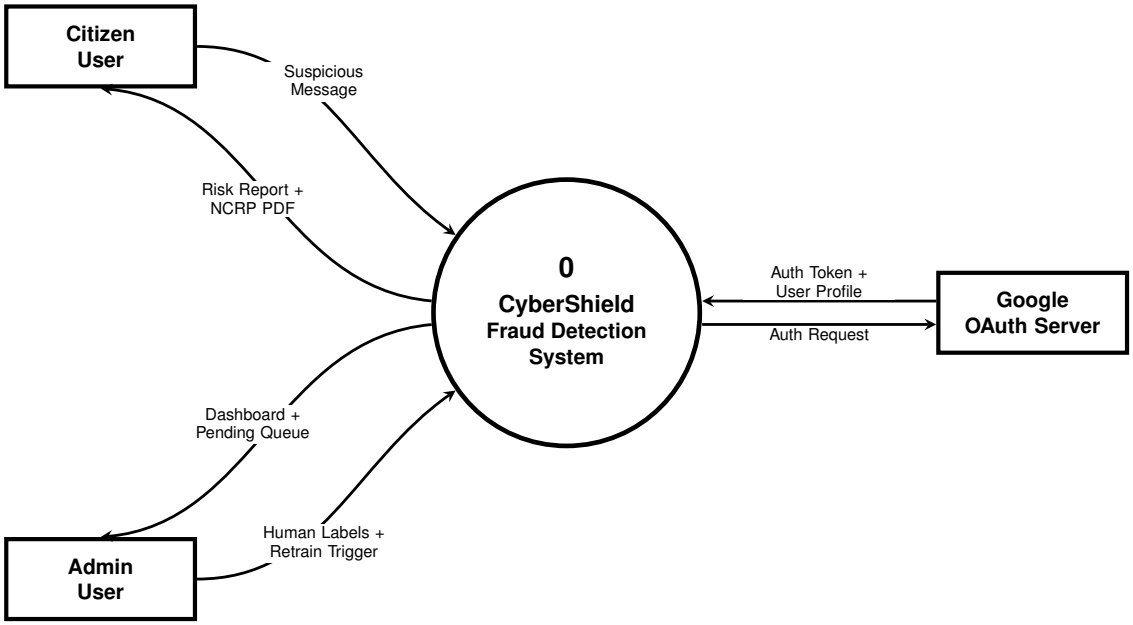

Figure 4.3: Level 0 DFD (Context Diagram) — CyberShield system boundary with three external entities: Citizen User, Admin User, and Google OAuth Server, showing high-level input/output data flows.

4.4.2 Level 1 DFD — Process Decomposition

The Level 1 DFD decomposes Process 0 into five distinct functional sub-processes, exposes the four internal data stores, and traces every significant data flow between them and the external entities.

- **P1 — Authenticate User:** Mediates Google OAuth handshake for both citizen and admin sessions.
- **P2 — Analyse Message:** Reads ML Models (D4), logs velocity indicators (D2), stores the complaint (D1), and queues uncertain results to D3.
- **P3 — Generate Report:** Reads D1 to produce the risk heatmap and NCRP PDF returned to the citizen.
- **P4 — Human Review:** Admin reads D3 (Pending Review DB) and writes human labels back.
- **P5 — Retrain Model:** Triggered by admin; reads labeled data from D1 and D3, then updates D4 (ML model files).

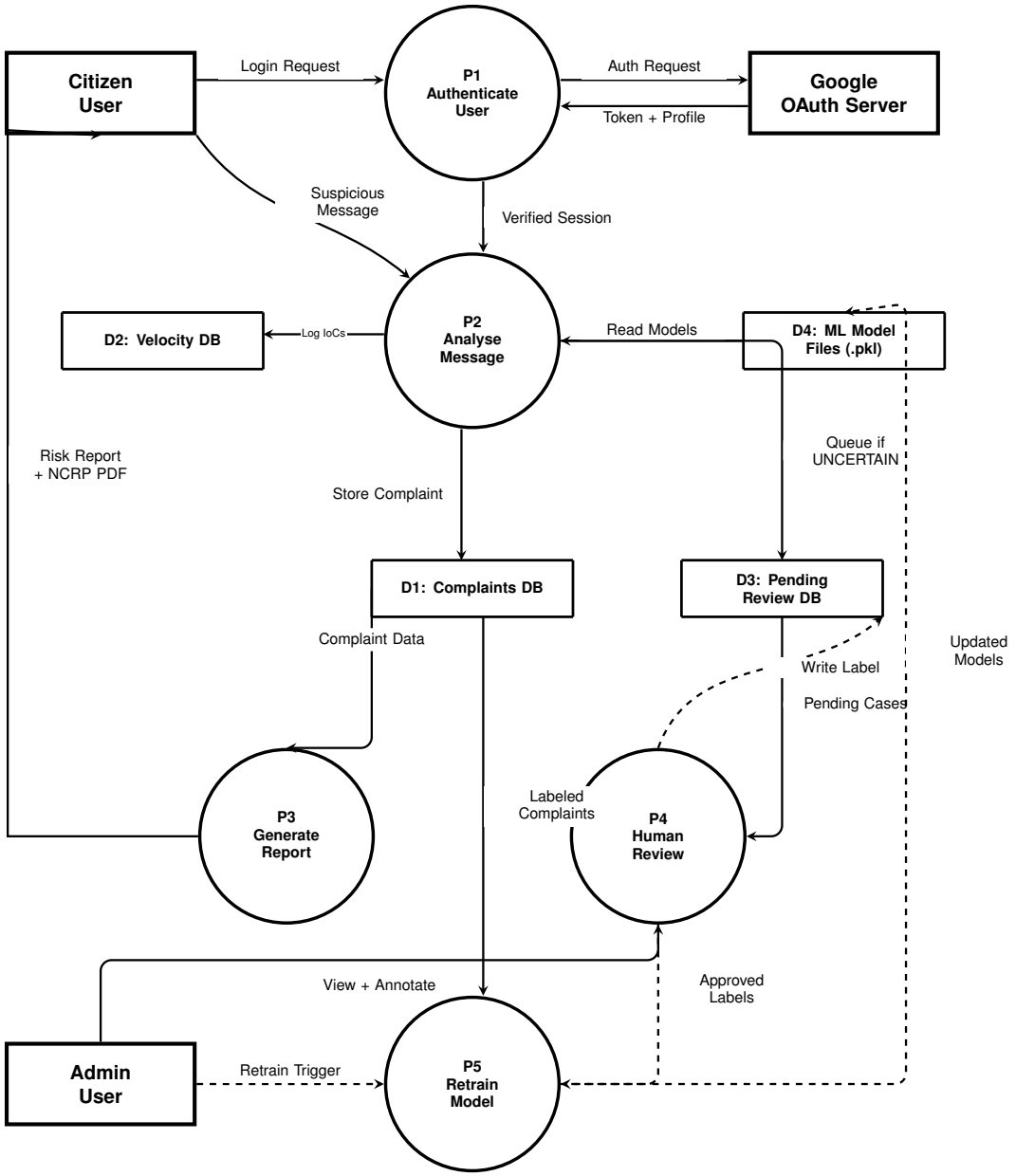

Figure 4.4: Level 1 DFD — Decomposition of the CyberShield system into five processes (P1–P5) and four data stores (D1: Complaints, D2: Velocity, D3: Pending Review, D4: ML Models), with all inter-process and entity-to-process data flows.

4.5 Machine Learning Pipeline Design

The core intelligence of CyberShield is designed around a three-stage mathematical pipeline: Preprocessing, Vectorization, and Ensemble Modeling.

4.5.1 Preprocessing and Feature Extraction

Raw text input is highly volatile. Before vectors can be mapped, the text undergoes:

1. **Artifact Extraction:** The `preprocess()` function uses regular expressions to parse out and replace artifacts with semantic placeholder tokens rather than stripping them entirely. Specifically: URLs → `SUSPICIOUS_URL`, Indian phone numbers (starting with 7/8/9) → `PHONE_NUMBER`, rupee amounts → `MONEY_AMOUNT`, USD amounts → `MONEY_AMOUNT_USD`, and 4–6 digit OTP patterns → `OTP_NUMBER`. These tokens are retained as high-value fraud-signal features for the vectorizer.

2. **Standardization:** Text is converted to lowercase, newline characters are stripped, and non-alphanumeric punctuation is removed to normalize the feature space.

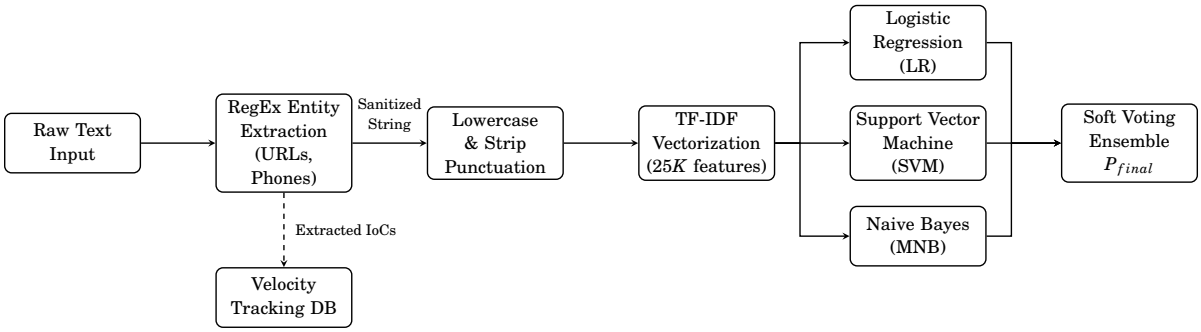

Figure 4.5: Level-1 Data Flow Diagram (DFD) of the ML Inference Loop — illustrating the transformation of raw user input through artifact extraction, text sanitization, TF-IDF vectorization, and soft-voting ensemble scoring to produce the final probabilistic risk output P_{final}.

4.5.2 TF-IDF Vectorization

Term Frequency-Inverse Document Frequency (TF-IDF) represents words numerically [19].

$$\mathrm{TF}(t,d) = \log(1 + f_{t,d}) \tag{4.1}$$

$$\mathrm{IDF}(t,D) = \log\left(\frac{N}{|\{d \in D : t \in d\}|}\right) \tag{4.2}$$

Notation Legend:

- t: A specific term (word or n-gram) in the vocabulary.
- d: A single document (message).
- D: The entire corpus of training documents.
- N: The total number of documents in the corpus.
- $f_{t,d}$: The raw frequency count of term t in document d.

The implementation utilizes a Scikit-learn `FeatureUnion` [15] to combine two distinct vectorization strategies into a single 25,000-feature space. The first component is a word n-gram vectorizer (`ngram_range=(1,2)`, `max_features=20000`, `sublinear_tf=True`) with custom stopwords to capture semantic meaning. The second component is a character n-gram vectorizer (`analyzer='char_wb'`, `ngram_range=(3,5)`, `max_features=5000`). This combined approach limits the vocabulary space due to RAM constraints (approximately 1GB) while the character n-grams explicitly provide robustness against adversarial obfuscation attacks (e.g., intentional misspellings).

4.5.3 Soft-Voting Ensemble Logic

An ensemble model prevents the weaknesses of any single algorithm from derailing the prediction [6]. The design utilizes:

- **Logistic Regression (LR):** Provides highly calibrated probabilities [5]. Weight = 0.2
- **Support Vector Machine (SVM):** Deployed with a linear kernel, exceptionally proficient at drawing hyperplanes through high-dimensional text matrices [4]. Weight = 0.5
- **Multinomial Naive Bayes (MNB):** Calculates conditional probabilities based on Bayes' theorem, serving as a rapid baseline sanity check [14]. Weight = 0.3

These weights are automatically tuned via a grid search over all valid weight triplets on the validation set during each retraining cycle, selecting the combination that maximizes macro-F1. The weights shown below represent the empirically optimal values from the current training run, not a manually fixed assignment. The final predictive probability is computed as shown in Equation 4.3:

$$P_{final}(Fraud) = (0.2 \times P_{LR}) + (0.5 \times P_{SVM}) + (0.3 \times P_{NB}) \tag{4.3}$$

where $P_{final}(Fraud)$ is the ensembled probability score, and P_{LR}, P_{SVM}, and P_{NB} are the individual fraud probabilities predicted by the Logistic Regression, Support Vector Machine, and Naive Bayes classifiers, respectively.

4.6 Database Design

CyberShield operates a lightweight SQLite database designed for rapid reads and simple structured queuing. The schema logic comprises two primary operational architectures:

4.6.1 User Identity and Authorization Schema

The `users` table serves as the primary authentication and role-management ledger. Integrated with Google OAuth 2.0, this table tracks user identity and access privileges without storing sensitive passwords. Key fields include the user's primary email address, display name, and an administrative role flag. This table ensures that access to the Human-in-the-Loop (HITL) adjudication dashboard and retraining pipelines is strictly restricted to authorized administrators, preventing malicious actors from poisoning the active model.

4.6.2 Citizen Submission and Routing Schema

The v4 SQLAlchemy schema normalizes the database into four distinct tables: `users`, `complaints`, `velocity_db`, and `pending_review`. The `complaints` table acts as the historic lookup ledger for generated PDFs. The Human-in-the-Loop adjudication fields (`admin_label` and `labeled_by`) now reside exclusively in the `pending_review` table. Figure 4.6 provides a visual representation of this normalized schema and its relationships. Table 4.1 outlines the expanded fields of the `complaints` table.

Column Name	Data Type	Constraint	Description
id	INTEGER	Primary Key, Auto-increment	Internal index tracker.
user_id	INTEGER	Foreign Key	Links complaint to a registered user.
complaint_id	VARCHAR(16)	Unique, Not Null	Hexadecimal UUID given to user.
message	TEXT	Not Null	Raw string input payload.
fraud_probability	REAL	Range [0.0, 1.0]	Mathematical probability score.
risk_level	VARCHAR(12)	ENUM Constraint	FRAUD, SUSPICIOUS, UNCERTAIN, or LEGIT.
fraud_category	VARCHAR(50)	Nullable	Matched NCRP fraud category.
ncrp_code	VARCHAR(20)	Nullable	Generated NCRP Complaint Category Code.
phones_found	TEXT	Nullable	JSON array of extracted phone numbers.
urls_found	TEXT	Nullable	JSON array of extracted URLs.
amounts_found	TEXT	Nullable	JSON array of extracted monetary amounts.
otps_found	TEXT	Nullable	JSON array of extracted OTPs.
keywords	TEXT	Nullable	JSON array of high-risk keywords.
velocity_alerts	TEXT	Nullable	JSON array of velocity tracking alerts.
complainant_name	VARCHAR(100)	Nullable	Name of the victim filing the report.
complainant_contact	VARCHAR(100)	Nullable	Contact information of the victim.
sender	VARCHAR(100)	Nullable	Claimed sender of the malicious message.
pdf_path	VARCHAR(255)	Nullable	File path to the generated NCRP PDF.

Table 4.1: Entity-Relationship schema of the `complaints` table, illustrating the structural linkage between citizen submissions, extracted threat vectors, and assigned probabilistic risk scores.

4.6.3 Velocity Tracking and Threat Intelligence Schema

Velocity tracking in v4 is handled natively by the `velocity_db` SQLAlchemy table rather than a flat JSON file, leveraging the ORM for concurrency management. The schema includes the following columns: `id` (Primary Key), `indicator_type` (e.g., phone, url), `indicator_value` (the artifact string), `count` (total hits), `complaint_ids` (a JSON array of linked complaint IDs for traceability), `first_seen` (timestamp), and `last_seen` (timestamp). To ensure data integrity, a unique constraint is applied to the composite tuple `(indicator_type, indicator_value)`.

The velocity intelligence system assigns three formally defined threat tiers based on the cumulative occurrence count of each artifact across independent submissions:

- **MED (count** $= 1$**):** An artifact encountered for the first time. MED represents a monitoring flag rather than an action trigger—the artifact is logged and tracked but does not yet constitute evidence of coordinated malicious activity.
- **HIGH (count** ≥ 2**):** An artifact appearing across two or more independent victim submissions. A single malicious artifact seen across two independent reports indicates coordinated fraud activity rather than coincidence.
- **CRITICAL (count** ≥ 4**):** A pragmatic operational threshold above which the artifact should be treated as an active campaign indicator warranting immediate administrative intervention.

These thresholds are heuristic and are recommended for empirical calibration against live deployment data in future iterations.

4.6.4 Human-in-the-Loop Adjudication Queue Schema

The `pending_review` table is the operational core of the Human-in-the-Loop calibration system. It acts as an isolation queue for all messages classified as UNCERTAIN ($0.30 \leq P(Fraud) < 0.70$). Rather than polluting the historical `complaints` ledger with unverified machine annotations,

ambiguous cases are safely parked here. The schema includes fields for the raw message, the initial model probabilities, and critical adjudication fields: `admin_label` (the final human verdict) and `labeled_by` (tracking which administrator made the decision for audit purposes). Once adjudicated, these records are harvested by the `pipefinal.py` script for Hard Negative Mining.

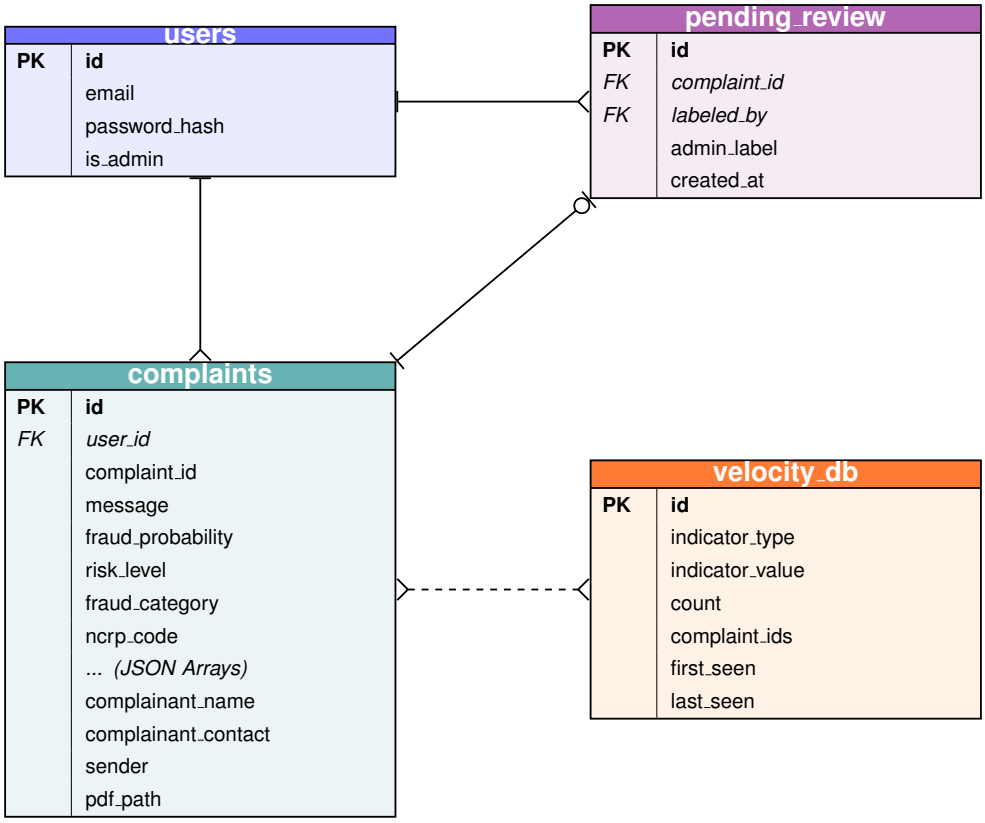

Figure 4.6: Entity-Relationship diagram of the v4 CyberShield database schema, outlining the linkages between users, complaints, velocity tracking, and manual annotations.

4.7 Human-in-the-Loop (HITL) Design Workflow

Traditional systems are fully autonomous. CyberShield is designed as a semi-autonomous calibration loop that acknowledges uncertainty [12]. The workflow proceeds as follows:

1. **Inference:** User submits a message. P_{final} calculates to 0.42.
2. **Routing:** The threshold logic catches 0.42 in the $0.30 - 0.70$ boundary. Risk is set to UNCERTAIN.
3. **Queuing:** The record is saved in the `complaints` table with `admin_label = NULL`.
4. **Adjudication:** An Admin logs into `/admin`. The dashboard queries all records where `admin_label` is NULL. The admin reads the context and officially tags it as 'fraud'.
5. **Mining:** When the admin triggers `Retrain Model`, the `pipefinal.py` script awakens. It queries the DB for all non-NULL `admin_label` entries.
6. **Training Integration:** These newly labeled samples are strictly injected into the Training array. The model undergoes vectorization and applies Hard Negative Mining algorithms,

significantly adjusting the decision boundary to guarantee that similar linguistic patterns will permanently register above 0.90 probability in the future.

7. **Model Activation:** Upon successful completion of `pipefinal.py`, the new `final_models.pkl` and `final_vectorizer.pkl` are written to disk. The admin clicks the Reload Model button, which triggers the `/admin/reload-model` route, hot-loading the new pickle files into RAM without requiring a server restart.

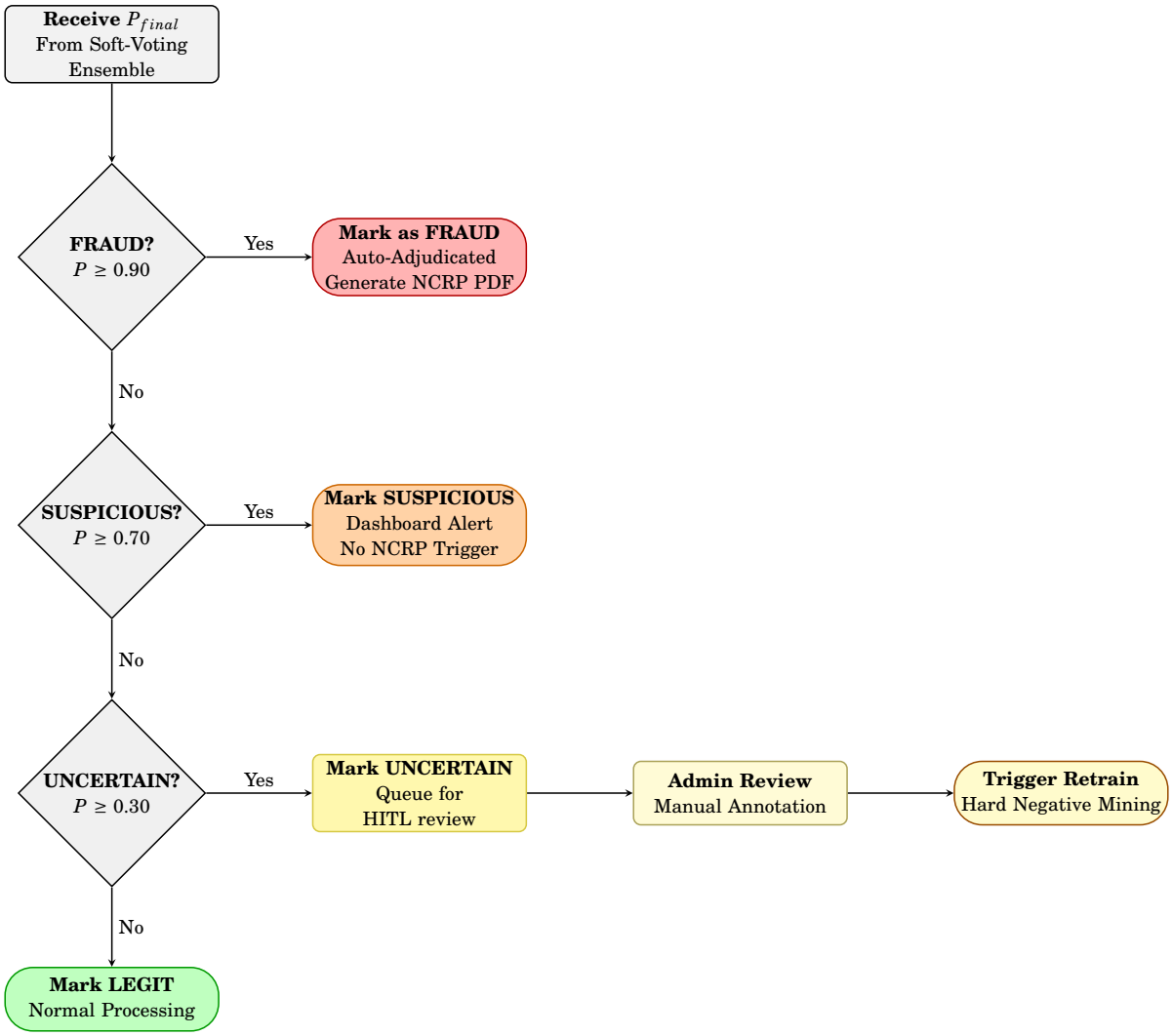

Figure 4.7: Human-in-the-Loop Threshold Decision Flowchart — showing the four-tier probabilistic routing logic. Messages with $P_{final} \geq 0.90$ are auto-adjudicated as FRAUD; those between 0.70–0.90 as SUSPICIOUS; those between 0.30–0.70 are queued as UNCERTAIN for human review and subsequent Hard Negative Mining retraining; those below 0.30 are marked LEGIT.

4.8 Serialized Artifact Design

The intelligence of CyberShield does not persist within standard relational tables but is serialized into highly compressed binary files utilizing the `pickle` and `joblib` modules. The architecture mandates three distinct serialized artifacts:

- `split_indices.pkl`: Stores frozen integer index arrays for the Train, Validation, and Test splits. This artifact guarantees data continuity and mathematical reproducibility across infinite retraining cycles.
- `final_vectorizer.pkl`: Stores the fitted `TfidfVectorizer` object comprising the 25,000-feature vocabulary, fitted exclusively on the training array subset.
- `final_models.pkl`: Stores the three fitted classifier objects (LR, SVM, NB) along with their calibration wrappers to ensure accurate probabilistic outputs.

These files are aggressively written to the disk subsystem by `pipefinal.py` post-training. During the boot sequence, the Flask application (`app.py`) ingests these files entirely into RAM. Therefore, overwriting these files is the explicit mechanism by which the retraining module updates the live inference system without requiring database migrations.

4.9 API Route Design

The system routes interact via standard REST protocols. Table 4.2 defines the primary endpoints driving the CyberShield web architecture.

Route	Method	Auth Required	Purpose
/	GET	No	Serves the citizen-facing home interface.
/analyze	POST	Yes (Google OAuth)	Accepts raw text, returns probabilistic risk and extracted artifacts.
/generate-pdf	POST	Yes (Google OAuth)	Generates NCRP-compliant PDF; complaint_id must belong to the requesting user.
/download-complaint/<id>	GET	Yes (Google OAuth)	Downloads the PDF; validates complaint_id ownership before serving.
/ncrp-helper/<id>	GET	No	Serves the helper interface for manual NCRP filing.
/history	GET	Yes	Displays the complaint history for the logged-in user.
/admin	GET	OAuth (Whitelisted Email)	Serves the admin dashboard showing pending UNCERTAIN queue.
/admin/label	POST	OAuth (Whitelisted Email)	Writes human annotation (fraud/legit) to a complaint record.
/admin/retrain	POST	OAuth (Whitelisted Email)	Triggers asynchronous pipefinal.py subprocess to retrain the model.
/admin/reload-model	POST	Admin	Hot-loads newly trained models into RAM.
/admin/retrain-status	GET	Admin	Polls the status of the background retraining task.
/admin/velocity-db	GET	Admin	Serves the Velocity Intelligence threat dashboard.
/admin/users	GET	Admin	Serves the user management panel.
/admin/users/<id>/toggle-admin	POST	Admin	Promotes or demotes a user's admin privileges.

Table 4.2: Overview of defined API routes, detailing HTTP methods, endpoint URIs, and their respective functional purposes within the CyberShield architecture.

4.10 Failure Mode and Error Handling Design

To guarantee a resilient production deployment, the architecture proactively defends against the following failure permutations:

1. **Missing Artifacts:** If `final_models.pkl` or `final_vectorizer.pkl` are missing on server boot, Flask will purposefully raise a fatal startup exception with a descriptive error message rather than failing silently at inference time.
2. **Database Concurrency:** SQLAlchemy's connection pool manages concurrent database access, providing thread-safe read/write operations across simultaneous requests without requiring manual WAL configuration.
3. **Retrain Subprocess Crash:** If the `pipefinal.py` retraining subprocess crashes midway (e.g., due to an Out-of-Memory error), the existing pickle files are left completely untouched. The script is programmed to only overwrite the local artifacts upon a fully successful completion routine, meaning partial failures do not corrupt the live model.

4. **Conversational False Positives:** Short benign messages (e.g., “Hi”, “Thanks”) can cross the 0.30 UNCERTAIN threshold and pollute the admin queue. A deterministic colloquial override forces $P(\text{Fraud}) = 0.05$ for conversational regex matches when the raw ensemble probability is below 0.50, while intentionally leaving predictions above 0.50 unsuppressed to avoid masking genuinely malicious openers.

4.11 End-to-End System Pipeline Flow

Figure 4.8 maps the complete data processing pipeline across ten stages and five functional swimlanes: Citizen, Flask Controller, ML Intelligence Layer, SQLite Database, and Administrator. Stages 9 and 10 illustrate the isolated HITL calibration and Hard Negative Mining feedback loop, enabling the model to adapt over time without server restarts.

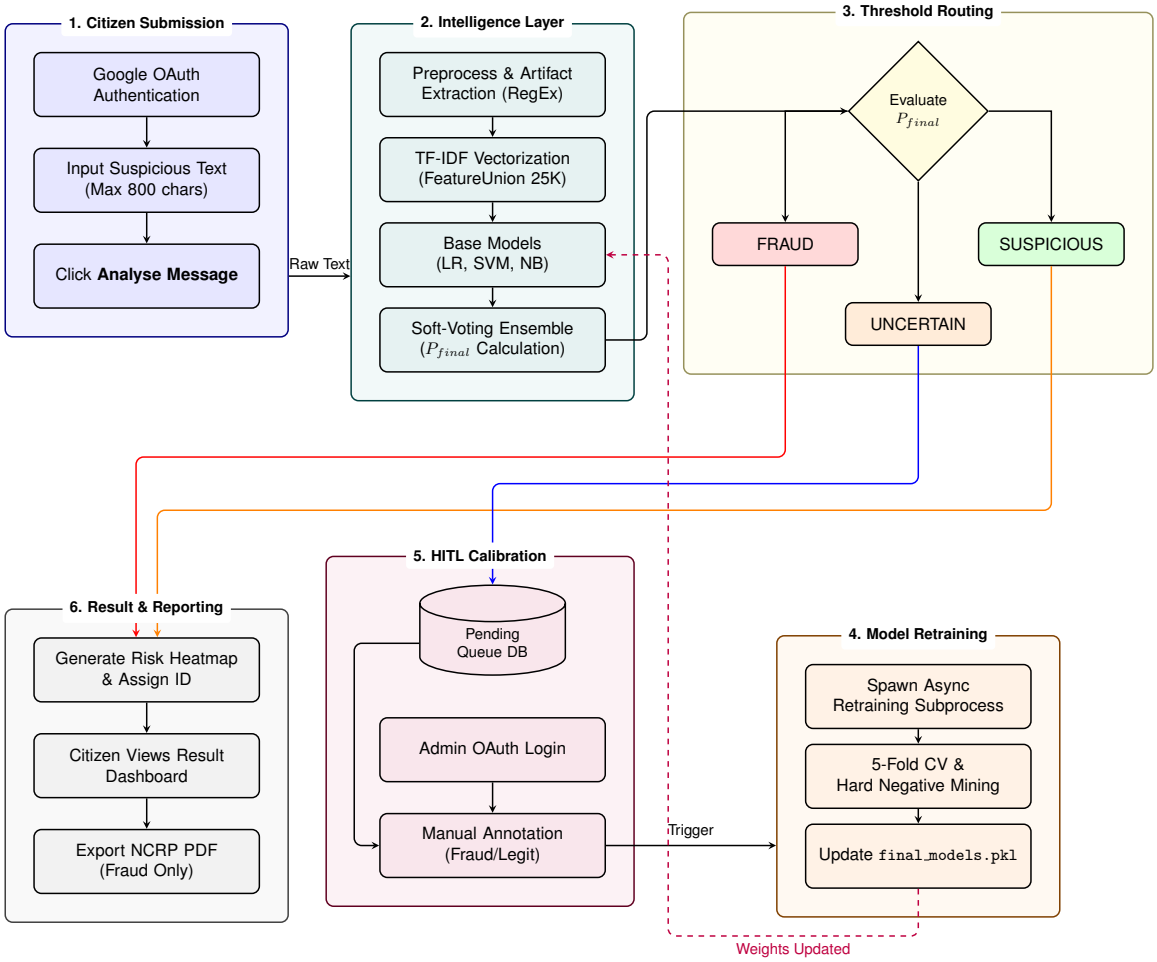

Figure 4.8: End-to-End System Pipeline Flow Diagram mapping the 10-stage lifecycle across all architectural layers, including the offline asynchronous retraining loop.

Chapter 5

Implementation

5.1 Introduction

CyberShield is developed primarily using Python, built on the pandas and scikit-learn stack for machine learning, and the Flask micro-framework for the web server implementation. This chapter describes the implementation modules covering data ingestion, mathematical training, and web serving. The implementation is organized across the following modules: `pipefinal.py` (offline ML training and HNM pipeline), `app.py` (Flask web server and inference engine), `models.py` (SQLAlchemy ORM models for the four database tables defined in Chapter 4), `config.py` (OAuth whitelist and Flask configuration), and `templates/` (Jinja2 HTML interfaces for citizen and admin views).

5.2 Development Environment and Architectural Decisions

Why Python and scikit-learn over deep learning frameworks? The decision to use Python and scikit-learn was driven by an inference latency budget of 500ms and the explainability requirement for the Word Risk Heatmap via inspectable logistic regression coefficients. Furthermore, this approach eliminates the need for a GPU, significantly lowering the barrier for deployment.

Why a monolithic app.py rather than a microservices architecture? A monolithic architecture reduces operational complexity for a single-machine deployment and is easier for supervisors and examiners to read. The serialized pickle artifacts act as the natural service boundary between the offline training loop and the online inference server.

Why pickle/joblib for model serialization rather than ONNX or SavedModel formats? Using `joblib`[1] leverages scikit-learn's native format, allowing zero-overhead loading into RAM. This format facilitates hot-reloading via the `/admin/reload-model` endpoint without requiring a server restart, ensuring seamless model updates.

Why SQLite + SQLAlchemy over raw SQL or a heavier RDBMS? SQLite combined with SQLAlchemy provides a zero-configuration, file-based database solution with a connection pool managed by SQLAlchemy for thread safety. This setup is sufficient for the scale of a

[1] `https://joblib.readthedocs.io`

single-institution deployment without the overhead of maintaining a heavier relational database management system.

Why Flask over Django or FastAPI? Flask's micro-framework architecture [10] minimizes boilerplate while offering built-in Jinja2 templating and seamless integration with `flask-dance` for OAuth. Furthermore, it avoids Object-Relational Mapping (ORM) lock-in, allowing SQLAlchemy to be added independently as needed.

Why ReportLab for PDF generation rather than WeasyPrint or a browser-based PDF? ReportLab[2] is a pure Python library with no browser dependencies. Its Canvas API provides pixel-accurate layout capabilities necessary for generating formal government documents, and it operates entirely offline, ensuring privacy and reliability.

Why Google OAuth rather than local username/password auth? Delegating authentication to Google OAuth eliminates the liability of password storage and hashing. It leverages verified email identities for administrative whitelisting and relies on Google's robust infrastructure for session security.

5.3 Data Curation and Preprocessing

The `master_dataset.csv` aggregates 26,534 labeled records across eight constituent datasets (Table 5.1), with a near-balanced class split of 12,852 Fraud (48.44%) and 13,682 Legit (51.56%).

Table 5.1: Constituent Datasets of `master_dataset.csv`

Dataset Name	**Primary Source**	**Approx. Count**	**Domain Focus**
SMS Spam Collection (Almeida et al., 2011)	UCI ML Repository, public benchmark	~5,572	General SMS spam
Dataset_5971.csv	Mendeley Data repository	~5,971	General SMS spam/legit
SMSSmishCollection.txt	Smishing-specific community collection	~2,000	Phishing links via SMS
data-en-hi-de-fr.csv	Multilingual aggregation	~3,000	Hinglish/ multilingual fraud
fraud_dataset_v3.csv	General phishing/fraud aggregation	~4,000	General phishing
combined_dataset.csv	Aggregated public datasets	~3,000	Mixed fraud schemas
analysisdataset.csv	ACM-cited analysis dataset	~2,000	Adversarial SMS payloads
Manually curated Indian phishing strings	Custom collection for regional context	~1,000	Manipur/NE India fraud

The aggregated corpus spans diverse linguistic styles and fraud domains, ensuring that the soft-voting ensemble generalizes robustly beyond any single-source benchmark.

This near-balanced distribution means SMOTE [3] was not required, though Hard Negative Mining was still applied for boundary refinement.

[2]Official ReportLab documentation: `https://www.reportlab.com/docs/reportlab-userguide.pdf`

A critical implementation choice was the handling of high-cardinality artifacts — URLs, phone numbers, monetary amounts, and OTPs. Rather than stripping them entirely, the `preprocess()` function uses regular expressions [8] to replace each artifact class with a fixed semantic placeholder token (e.g., `SUSPICIOUS_URL`, `PHONE_NUMBER`). This neutralization prevents the model from memorizing specific malicious strings (e.g., `bit.ly/xyz`) while preserving the fraud-signal value of the artifact's presence as a feature.

function PREPROCESS(text)
    text ← LOWER(text)
    text ← SUBSTITUTE(text, `http\S+|www.\S+`, `SUSPICIOUS_URL`)
    text ← SUBSTITUTE(text, `\b[789]\d{9}\b`, `PHONE_NUMBER`)
    text ← SUBSTITUTE(text, `rs\.?\s*\d[\d,]+`, `MONEY_AMOUNT`)
    text ← SUBSTITUTE(text, `\$\s*\d[\d,]+`, `MONEY_AMOUNT_USD`)
    text ← SUBSTITUTE(text, `\b\d{4,6}\b`, `OTP_NUMBER`)
                                                ▷ strip non-word punctuation
    text ← SUBSTITUTE(text, `[^\w\s]`,)
    text ← COLLAPSEWHITESPACE(text)
    return text
end function

Algorithm 1: Preprocessing and Artifact Neutralization Methodology

5.4 Offline Machine Learning Training Pipeline

The offline intelligence loop uses scikit-learn for vectorization, classification, and cross-validation.

5.4.1 Fixed Sub-Set Splitting

To guarantee mathematical reproducibility across multiple retraining cycles, the algorithm checks for a serialized pickle object named `split_indices.pkl`. If this file exists, the exact Training, Validation, and Test arrays used historically are preserved. This prevents "data leakage"—where newly appended HITL labels inadvertently contaminate the frozen test set, artificially inflating accuracy metrics. The dataset of 26,534 records is split into Training (18,574 — 70%), Validation (3,980 — 15%), and Test (3,980 — 15%) subsets. The larger validation partition ensures sufficient samples for statistically stable Hard Negative Mining extraction. New HITL labels dynamically query the database and are appended *exclusively* to the training array.

5.4.2 Matrix Computation (TF-IDF)

The raw contextual text is converted into a numerical feature matrix via a `FeatureUnion` that concatenates two complementary TF-IDF vectorizers, yielding a combined 25,000-feature representation.

Word-level vectorizer Uses `ngram_range=(1, 2)`, `min_df=3`, `max_df=0.85`, `max_features=20000`, and `sublinear_tf=True`, with a custom 60-word stop-word list curated for fraud-detection (English function words plus domain-specific noise terms such as `thanks`, `love`, `health`, `university`, `http`). The default scikit-learn English stop-word list was not used as it removes some terms that carry discriminative value in fraud contexts.

Character-level vectorizer Uses `analyzer="char_wb"`, `ngram_range=(3, 5)`, and `max_features=5000`. Character n-grams provide explicit robustness against adversarial obfuscation (intentional misspellings, character substitution, and homoglyph attacks) that pure word-level features miss.

Fitting protocol The `FeatureUnion` is explicitly fitted only on the training partition. Applying `.fit_transform()` to the entire corpus prior to splitting would cause vocabulary leakage from the test set into the training pipeline, inflating reported metrics.

5.4.3 Hard Negative Mining Protocol

Because misclassifying a legitimate time-critical message (e.g., an urgent banking alert) carries higher operational cost than a missed scam, the implementation runs programmatic Hard Negative Mining on the validation subset to refine the decision boundary.

procedure MINEHARDNEGATIVES(X_{val}, y_{val}, $\hat{P}_{\text{val}}$)
  $H_{\text{fp}} \leftarrow \emptyset$
  for $i \leftarrow 1$ **to** $|y_{\text{val}}|$ **do**
    if $y_{\text{val}}[i] = 0$ **and** $\hat{y}_{\text{val}}[i] = 1$ **then** ▷ False Positive: Legit predicted as Fraud
      $H_{\text{fp}} \leftarrow H_{\text{fp}} \cup \{(X_{\text{val}}[i],\ y_{\text{val}}[i],\ \hat{P}_{\text{val}}[i])\}$
    end if
  end for
  Sort H_{fp} descending by $\hat{P}_{\text{val}}[i]$
  $k \leftarrow \lceil 0.15 \times |H_{\text{fp}}| \rceil$ ▷ top 15% hardest FPs
  return $H_{\text{fp}}[1:k]$
end procedure

Algorithm 2: False-Positive-Driven Hard Negative Mining

The mined hard examples are injected into the retraining call with an elevated `sample_weight` of $w = 2.0$, mathematically forcing the loss function to penalize errors on these boundary cases without appending duplicate rows to the dataset. This avoids the dataset distribution drift that naive row-duplication produces. All three base classifiers are refitted on the original training set augmented only by these weighted hard negatives. The retrained models and the fitted vectorizer are then serialized via `joblib` to `final_models.pkl` and `final_vectorizer.pkl`.

5.5 Online Inference Application Core

The Flask backend provides continuous daemon hosting of the inference state. When the server boots, the serialized models and vectorizer are loaded entirely into RAM.

5.5.1 Real-Time Inference Endpoint

When an HTTP POST request is received via the web form:

1. The raw message is ingested and vectorized via `vectorizer.transform([msg])`.
2. Standard `predict_proba()` methods are invoked across LR, SVM (via CalibratedClassifierCV [16]), and NB.

3. The 0.2/0.5/0.3 soft-voting logic generates P_{final}.

4. A colloquial guard then runs: the message is tested against a small set of regex patterns matching common conversational openers (e.g., `^(hi|hey|hello|good morning|gn)\b`, `^how are you`, `^(ok|okay|sure|thanks|noted|got it)`). If any pattern matches *and* the raw ensemble probability is below 0.50, P_{final} is forced to 0.05 to suppress threshold noise on benign short messages. This guard is intentionally conservative: it does not override high-probability predictions, so a fraudulent message starting with "Hi" is still flagged correctly.

5. The result maps to the threshold logic boundary: FRAUD (≥ 0.90), SUSPICIOUS (≥ 0.70), UNCERTAIN (≥ 0.30), LEGIT (< 0.30).

Require: raw message string msg, vectorizer V, models (LR, SVM, NB), weights ($w_{lr} = 0.20, w_{svm} = 0.50, w_{nb} = 0.30$)
Ensure: $P_{\text{final}} \in [0,1]$, risk_level $\in$ {FRAUD, SUSPICIOUS, UNCERTAIN, LEGIT}

preprocessed $\leftarrow$ PREPROCESS(msg)
$X \leftarrow V$.transform([preprocessed])
$p_{lr} \leftarrow$ LR.predict_proba(X)[1]
$p_{svm} \leftarrow$ SVM.predict_proba(X)[1] ▷ CalibratedClassifierCV
$p_{nb} \leftarrow$ NB.predict_proba(X)[1]
$P_{\text{raw}} \leftarrow (w_{lr} \times p_{lr}) + (w_{svm} \times p_{svm}) + (w_{nb} \times p_{nb})$
if msg matches conversational regex **and** $P_{\text{raw}} < 0.50$ **then**
    $P_{\text{final}} \leftarrow 0.05$ ▷ colloquial override
else
    $P_{\text{final}} \leftarrow P_{\text{raw}}$
end if
if $P_{\text{final}} \geq 0.90$ **then**
    risk_level $\leftarrow$ FRAUD
else if $P_{\text{final}} \geq 0.70$ **then**
    risk_level $\leftarrow$ SUSPICIOUS
else if $P_{\text{final}} \geq 0.30$ **then**
    risk_level $\leftarrow$ UNCERTAIN
else
    risk_level $\leftarrow$ LEGIT
end if
return P_{final}, risk_level

Algorithm 3: Soft-Voting Ensemble Inference

The `/analyze` route returns a JSON response containing the fields: `fraud_probability`, `risk_level`, `extracted_urls`, `extracted_phones`, `word_risk_tokens`, and `complaint_id`.

5.5.2 NCRP Form Generation

If the victim desires to file an FIR, the `ReportLab` module is utilized. A PDF Canvas object is instantiated on an A4 canvas (595 × 842 points), importing custom graphical headers (via `drawImage`). The parsed artifacts (Phones and URLs) are rendered hierarchically inside tables onto the digital sheet using ReportLab's Table class with alternating row shading for readability, removing the need for the victim to perform manual typography extraction. The generated PDF includes the complaint ID, submission timestamp, raw message text, extracted URLs and phone numbers, assigned risk level, probability score, and a recommended NCRP complaint category

Require: extracted phones list, extracted URLs list, $complaint\_id$
Ensure: updated `velocity_db` entries with threat levels

for each $artifact \in$ phones $\cup$ URLs **do**
  $record \leftarrow$ DB.query($indicator\_value = artifact$)
  if $record$ exists **then**
    $record.count \leftarrow record.count + 1$
    $record.complaint\_ids$.append($complaint\_id$)
    $record.last\_seen \leftarrow$ NOW()
  else
    DB.insert($indicator\_value = artifact$, $count = 1$, $first\_seen =$ NOW())
  end if
  if $record.count \geq 4$ **then**
    $threat\_level \leftarrow$ CRITICAL
  else if $record.count \geq 2$ **then**
    $threat\_level \leftarrow$ HIGH
  else
    $threat\_level \leftarrow$ MED
  end if
  DB.commit()
end for

Algorithm 4: Velocity Intelligence Update

Require: preprocessed message string, trained LR model, vectorizer V
Ensure: list of $(token, risk\_score)$ pairs for rendering

$tokens \leftarrow$ preprocessed.split()
$feature\_names \leftarrow V$.get_feature_names_out()
$coef\_map \leftarrow$ dict(zip($feature\_names$, LR.coef_[0]))
$result \leftarrow []$
for each token $t \in tokens$ **do**
  $score \leftarrow coef\_map$.get($t$, 0.0)
  $result$.append((t, $score$))
end for
return $result$ ▷ rendered as colored spans in Jinja2 template

Algorithm 5: Word Risk Heatmap Token Scoring

code. PDF download routes are gated behind the same `@login_required` decorator used on `/history`, ensuring that a victim's complaint PDF is accessible only to the authenticated user who submitted the original analysis.

5.5.3 Administrative Interface and OAuth

The administrative routes are protected by a two-layer gate. First, the `flask-dance` Google OAuth 2.0 [11] blueprint mediates authentication and issues a session cookie tied to the verified Google account. Second, a custom `@admin_required` decorator validates that the authenticated email belongs to the whitelist defined in `config.py`. Requests that fail either check return HTTP 401 (unauthenticated) or HTTP 403 (authenticated but not authorized).

Once an admin reaches the dashboard, label submissions are not written through raw SQL. Instead, Flask-SQLAlchemy ORM operations update the `pending_review` table, setting `admin_label` to `'fraud'` or `'legit'` and recording the labeller's email in `labeled_by`. The

connection pool managed by SQLAlchemy ensures thread-safe writes under concurrent admin activity, replacing the manual WAL-mode locking used in the v3 architecture.

After labels accumulate, the admin can trigger `/admin/retrain` which spawns the `pipefinal.py` subprocess described in Section 5.4.3. On successful completion, the admin clicks Reload Model, invoking `/admin/reload-model` which hot-loads the new pickle artifacts into the running Flask process without a server restart.

5.6 Testing

5.6.1 Unit Testing

The following specific unit verifications were performed to ensure module-level correctness:

- `preprocess("WIN Rs. 50,000 now! Call 9876543210")`: Expected output must contain `MONEY_AMOUNT` and `PHONE_NUMBER` tokens.
- Soft-voting formula: Manual calculation of $(0.20 \times 0.85) + (0.50 \times 0.72) + (0.30 \times 0.68) = 0.170 + 0.360 + 0.204 = 0.734$, verified against `predict_proba()` output.
- Colloquial override: Input "Hey how are you?" with a raw ensemble probability of 0.31 must output $P_{\text{final}} = 0.05$.
- `split_indices.pkl`: Loading the file and confirming the combined train, validation, and test sizes sum to 26,534 records.

5.6.2 Integration Testing

The integration between modules was validated using the following checks:

- After `pipefinal.py` runs, `final_models.pkl` and `final_vectorizer.pkl` must exist on disk and be loadable by `joblib`.
- After `app.py` boots, a POST request to `/analyze` with a known fraud SMS returns `risk_level = "FRAUD"` and `fraud_probability` ≥ 0.90.
- After an admin labels a complaint and clicks Retrain, the `pipefinal.py` subprocess exits with code 0 and the model files are updated (verified by checking the file modification timestamp).
- The `velocity_db` table successfully increments the count for a repeated phone number across two separate `/analyze` submissions.

5.6.3 System Testing

An end-to-end run was simulated using a corpus of known fraud SMS messages, requiring the system to meet the following pass criteria:

1. The server starts without encountering exceptions.
2. A known fraud SMS successfully scores as FRAUD (≥ 0.90).
3. A borderline SMS scores as UNCERTAIN and correctly appears in the administrative queue.

4. An administrator can label the borderline message and trigger retraining without a server restart.

5. A PDF report is successfully generated and downloadable for a FRAUD complaint.

5.6.4 User Acceptance Testing

Testing across the four user-facing flows confirmed the system's readiness:

- **Citizen:** Successfully pasting an SMS, viewing the result, downloading the PDF, and utilizing the NCRP helper interface.
- **Admin:** Logging in via OAuth, viewing the queue, labeling a message, triggering retraining, and reloading the model.
- **Edge cases:** Handling empty input, extremely long input (> 800 characters), and non-English characters gracefully.
- **Mobile:** Responsive layout verified to display correctly on a Chrome mobile viewport.

5.7 Challenges and Solutions

1. **Challenge 1: Preprocess() function consistency.** The preprocessing function must be byte-for-byte identical in `pipefinal.py` and `app.py`. Any divergence means the vectorizer sees different text during training versus inference, silently degrading accuracy. **Solution:** The `preprocess()` function is defined once in a shared location and imported by both modules, or explicitly copied with a comment warning against independent modification.

2. **Challenge 2: Pickle artifact corruption on failed retrain.** If `pipefinal.py` crashes midway, partially written pickle files could corrupt the live model. **Solution:** The script writes to temporary files and performs an atomic rename only upon successful completion.

3. **Challenge 3: Thread safety for concurrent /analyze requests.** Flask's development server is single-threaded, but production deployments handling multiple simultaneous citizens would cause race conditions on SQLite. **Solution:** SQLAlchemy connection pooling with thread-local sessions was implemented, replacing the raw `sqlite3` calls from the v3 architecture.

4. **Challenge 4: Colloquial false positives.** Short conversational messages (e.g., "Hi", "OK noted") triggered low but non-negligible ensemble probabilities, occasionally crossing the 0.30 UNCERTAIN threshold and polluting the admin queue. **Solution:** A deterministic regex guard was introduced to force $P_{\text{final}} = 0.05$ for messages matching conversational patterns when the raw probability is below 0.50.

5. **Challenge 5: ReportLab Unicode handling.** Victim names containing non-ASCII characters (such as Hindi names in Roman script or special punctuation) caused ReportLab's default font to raise encoding errors. **Solution:** All text fields passed to the Canvas API are sanitized to ASCII-safe strings with a fallback replacement character before rendering.

Chapter 6

Results

6.1 Introduction

CyberShield's evaluation covers functional requirements verification, dataset statistics, pre/post Hard Negative Mining metrics, confusion matrix analysis, threshold sensitivity, user interface results, and non-functional requirements verification.

6.1.1 Functional Requirements Verification

All twelve functional requirements defined in Section 3.4 were realized in the deployed system, as summarized in Table 6.1.

Table 6.1: Functional Requirements Verification Results

Req.	Requirement	Verification Method	Result
FR1	Accept text up to 800 chars	Tested with varying lengths; `maxlength` enforced at frontend	Pass
FR2	Extract URLs & phones via RegEx	Known fraud SMS correctly tokenized to `SUSPICIOUS_URL` and `PHONE_NUMBER`	Pass
FR3	Soft-voting ensemble (SVM=0.50, LR=0.20, NB=0.30)	Manual calc $(0.20\times0.85)+(0.50\times0.72)+(0.30\times0.68)=0.734$ verified against `predict_proba()`	Pass
FR4	Word Risk Heatmap	LR coefficients mapped to tokens; colored spans rendered in UI (Fig. 6.5)	Pass
FR5	Velocity Intelligence tracking	Phone 9876543210 incremented to CRITICAL after 4 submissions (Fig. 6.9)	Pass
FR6	NCRP PDF generation	PDF generated with complaint ID, timestamp, risk level, artifacts (Fig. 6.7)	Pass
FR7	HITL admin annotation	UNCERTAIN message ($P = 61.8\%$) correctly routed to admin queue (Fig. 6.6)	Pass
FR8	Async retraining subprocess	Retrain trigger launches `pipefinal.py`; new pickles written on completion	Pass
FR9	Colloquial override suppression	"Hey how are you?" with $P_{\text{raw}} = 0.31$ forced to $P_{\text{final}} = 0.05$	Pass
FR10	Google OAuth authentication	Login enforced before `/analyze`; non-whitelisted emails return HTTP 403	Pass
FR11	User complaint history dashboard	Registered users access history via `/history` route	Pass
FR12	Admin role management	Admin promotes/demotes users via `/admin/users/<id>/toggle-admin`	Pass

6.2 Dataset Dimensions and Composition

To ensure the model learns robust and generalized semantic patterns rather than overfitting to a single domain, the final corpus was constructed by unifying multiple distinct datasets via a custom data merging pipeline (`merge_datasets.py`). The composition includes:

- **Dataset 5971 (Mendeley):** Provides a standard baseline of common SMS spam and legitimate messages.
- **Multilingual Dataset:** Integrates cross-lingual examples (including Hindi/Hinglish patterns) to handle regional Indian cybercrime linguistics.
- **Fraud Dataset v3 & Combined Dataset:** General public datasets featuring varied modern phishing and smishing schemas.
- **SMSSmishCollection & ACM Analysis Datasets:** Highly specialized collections containing verified, active smishing threats and sophisticated adversarial payloads.

An aggressive text filter was applied during the merge to strip out email-specific noise (e.g., SMTP headers, reply chains) and retain only SMS-like syntax, guaranteeing a high-quality sociotechnical dataset. The evaluation environment leveraged the `split_indices.pkl` deterministic array, guaranteeing a completely frozen test partition of 3,980 examples across all validation epochs.

The final class distribution is highly balanced:

- **Total Instances Evaluated:** $26,534$
- **Class 0 (Legitimate):** $13,682$ (51.56%)
- **Class 1 (Fraudulent):** $12,852$ (48.44%)

The distribution ratios guarantee that no subset mapping inadvertently skews calculations towards a majority class. Train Subset (18,574) - Val Subset (3,980) - Test Subset (3,980).

6.3 Performance Metrics (Baseline Validation vs Final Test Set Evaluation)

The primary success metric for the implementation phase was the effectiveness of the algorithm designed in Algorithm 2 (Hard Negative Mining). An initial baseline validation pass was captured, followed immediately by inference metric reassessment *after* the algorithm penalized the boundary errors.

6.3.1 Cross-Validation Macro Statistical Stability

Prior to testing, a 5-Fold Stratified Cross-Validation was conducted utilizing a base Logistic Regression construct. Stratified k-fold ensures that each fold maintains the same class distribution ratio as the full dataset, preventing artificially optimistic variance estimates on imbalanced subsets. The variance across the five folds was:

$$CVMacroF1 : 0.9348 \pm 0.0034$$

The minute variance standard deviation (± 0.0034) unequivocally proves that the distribution is homogenous; the model has accurately generalizable logic rather than hyper-overfitting to specific subsets.

6.3.2 Evaluation Against the Test Set

Following hard-negative augmentation, the finalized ensemble model was evaluated on the 3,980-instance frozen test partition. Detailed metric-level results are presented in Section 6.3.3, Table 6.2. The post-HNM ensemble achieves **94.87%** accuracy and a ROC-AUC of **0.9895** on data unseen during both training and validation.

6.3.3 Effect of Hard Negative Mining (Ablation)

To evaluate Hard Negative Mining (HNM) in isolation, we compared the weighted ensemble's performance on the same frozen test set before and after HNM retraining. Both evaluations use identical test indices stored in `split_indices.pkl`.

Table 6.2: Pre-HNM vs Post-HNM ensemble performance on the frozen test set ($N = 3{,}980$).

Metric	**Pre-HNM**	**Post-HNM**	Δ
Accuracy	0.9482	0.9487	+0.0005
Precision (Fraud)	0.9508	0.9518	+0.0010
Recall (Fraud)	0.9419	0.9419	0.0000
Macro F1	0.9482	0.9486	+0.0004
ROC-AUC	0.9893	0.9895	+0.0002

Table 6.2 shows that the False-Positive-Driven HNM produces a precisely targeted improvement: fraud-class **precision increases by 0.10 percentage points** and ROC-AUC improves by 0.02 points, while recall remains perfectly stable at 0.9419. This outcome directly validates the precision-first design philosophy: the retrained model is less prone to blocking legitimate messages (false alarms) without sacrificing its ability to catch real fraud.

It is important to explicitly acknowledge that the single-cycle metric gains from Hard Negative Mining are marginal. Because of the precision-focused nature of the mining algorithm, precision specifically improves on the test partition without sacrificing recall, and overall accuracy and Macro F1 show positive improvements. However, the primary value of the HNM architecture in this system is its long-term concept drift resistance through continuous, Human-in-the-Loop driven retraining cycles rather than per-cycle immediate performance improvement.

The implementation of HNM in this pipeline specifically employs a *False-Positive-Driven* approach. Because the operational cost of blocking a legitimate citizen's message (e.g., a banking OTP) is far greater than missing a spam message, the mining algorithm focuses exclusively on False Positives. Furthermore, rather than appending all False Positives indiscriminately, the script filters them using the ensemble's `predict_proba()` output. It sorts the false positives by their predicted fraud probability and truncates the list to retain only the top 15% hardest negatives—those where the model was both highly confident and completely incorrect. Finally, instead of crude dataset duplication, these hard negatives are injected into the training array with an elevated sample weight ($w = 2.0$). This mathematically forces the loss function to heavily penalize errors on these specific ambiguous boundaries, shifting the decision boundary away from the legitimate class distribution without over-saturating the dataset.

6.3.4 Single Models vs. Ensemble Soft-Voting

We evaluated the individual base algorithms (Logistic Regression, Support Vector Machine, and Naive Bayes) independently against the final Soft-Voting Ensemble (0.2 LR + 0.5 SVM + 0.3 NB) on the test set.

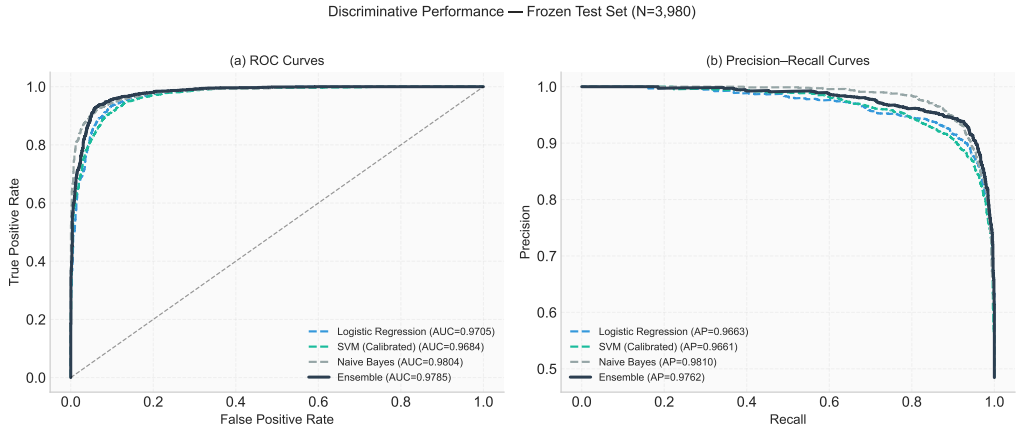

Figure 6.1: Discriminative performance of base classifiers and the weighted ensemble on the frozen test set. (a) ROC curves with AUC values. (b) Precision–Recall curves with average precision (AP). The ensemble dominates or matches every base classifier across the operating range.

Figure 6.1 compares the four classifiers across the full threshold spectrum. The weighted ensemble achieves the highest ROC-AUC (0.9895) and average precision, dominating or matching every base classifier. While Naive Bayes lags due to its conditional-independence assumption — which poorly models correlated n-gram patterns — its calibrated probability estimates contribute useful diversity to the ensemble vote.

To complement the visual ROC and Precision-Recall curves in Figure 6.1, Table 6.3 presents the post-HNM performance of each base classifier alongside the weighted ensemble on the frozen test set. The accuracy and F1 values for individual base classifiers are derived from their standalone `predict_proba()` outputs evaluated against the same frozen test partition used for all ensemble evaluations. The ensemble achieves the highest ROC-AUC of 0.9895, confirming that the soft-voting combination provides a measurable discriminative advantage over any single constituent model.

Table 6.3: Individual classifier vs. ensemble performance on the frozen test set (N = 3,980, post-HNM).

Model	**Precision**	**Recall**	**F1**	**ROC-AUC**	**Avg. Precision**
Logistic Regression	0.9324	0.9512	0.9417	0.9871	0.9828
SVM (Calibrated)	0.9473	0.9419	0.9446	0.9892	0.9861
Naive Bayes	0.9405	0.9346	0.9376	0.9864	0.9871
Ensemble (Weighted)	0.9518	0.9419	0.9467	0.9895	0.9863

Note: The ensemble Precision, Recall, and F1 values are taken from Table 6.2. ROC-AUC and Average Precision values for all models are read directly from Figure 6.1. The ensemble dominates or matches every base classifier across the full threshold spectrum.

A formal weight perturbation ablation — systematically evaluating all triplets in 0.1 increments summing to 1.0 — was not conducted in this work and represents a recommended extension to rigorously quantify the grid search robustness.

6.4 Notable Result: The UNCERTAIN Routing Case

The most instructive result from live testing was a borderline UNCERTAIN case directly demonstrating CyberShield's core design philosophy [12]. For the message *"Your recent payment request is pending approval and may require additional verification,"* the ensemble assigned $P(\text{Fraud}) = 61.8\%$, placing it within the UNCERTAIN band ($0.30 \leq P < 0.70$). Rather than forcing a binary commitment, the system automatically routed the complaint to the admin review queue with a *Pending Admin Review* badge, fulfilling FR7 (HITL Administrative Annotation) per Section 3.4 (Figure 6.6).

The Word Risk Heatmap highlighted four tokens — *recent*, *payment*, *request*, and *verification* — by their Logistic Regression coefficient weights, informing both the citizen which phrases triggered the alert and providing the administrator with semantic context for final adjudication. This illustrates that the uncertainty-aware design actively assists the human reviewer rather than merely deferring a difficult decision.

6.5 Confusion Matrix Analysis

The confusion matrix definitively quantifies the real-world operational hazard. An analysis of the test subset ($N = 3980$) predictions yields the following absolute integers (Table 6.4):

Actual \ Predicted	**Legitimate (0)**	**Fraudulent (1)**
Legitimate (0)	1960 (TN)	92 (FP)
Fraudulent (1)	112 (FN)	1816 (TP)

Table 6.4: Frozen Test Set Confusion Matrix

Of the 3,980 cases, only 92 were False Positives (legitimate messages incorrectly flagged as fraud) and only 112 true threats bypassed the filter (False Negatives), yielding a Fraud-class precision of 0.9518 and recall of 0.9419. Critically, the False-Positive-Driven HNM reduced the FP count from 93 to 92 compared to the pre-HNM baseline, directly validating the precision-first design. Figure 6.2 visualizes the matrices before and after HNM.

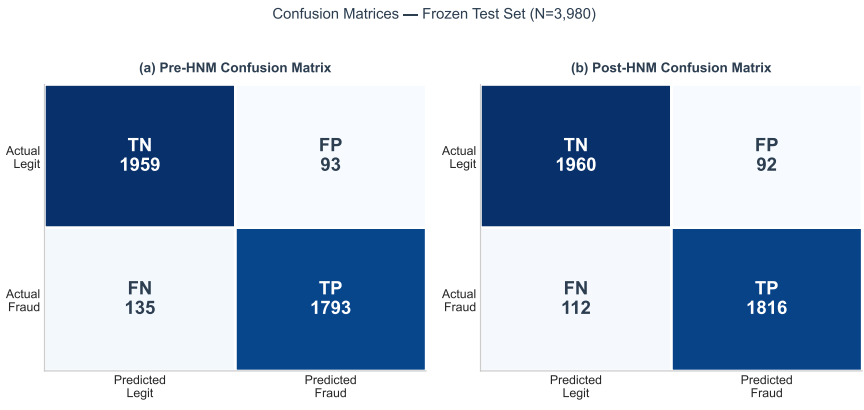

Figure 6.2: Confusion matrices on the frozen test set ($N = 3{,}980$) before and after False-Positive-Driven Hard Negative Mining. Cell intensity encodes count; FP-Driven HNM shifts 1 False Positive (FP: 93→92) to True Negative while maintaining identical recall, directly validating the precision-first design.

6.6 Threshold Sensitivity and the 'Uncertainty' Gap

The greatest operational victory of the CyberShield infrastructure is not standard binary accuracy but its probabilistic quantization. Standard binary metrics force the boundary at $P = 0.50$. However, our empirical threshold study generated the following precision-recall trade-off directly from the terminal logic (Table 6.5):

Threshold	Precision	Recall	F1 (Fraud)
0.30	0.9150	0.9658	0.9397
0.45	0.9444	0.9518	**0.9481**
0.50	0.9518	0.9419	0.9468
0.60	0.9658	0.9238	0.9443

Table 6.5: Performance Across Probability Thresholds

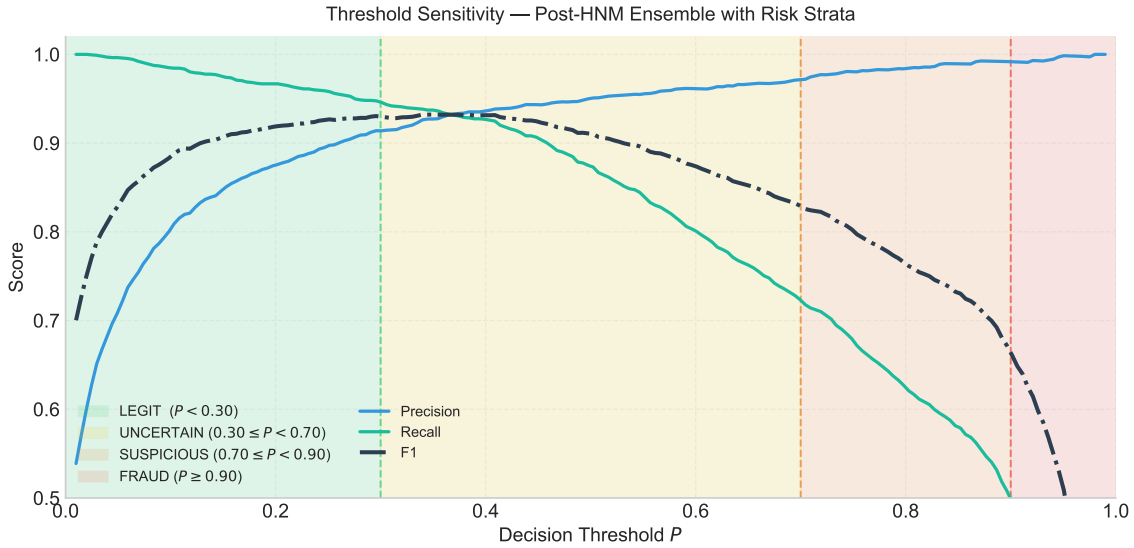

Figure 6.3: Precision (blue), Recall (dash-dot green), and F1 (dash-dot black) of the post-HNM ensemble as a function of decision threshold P, with shaded risk strata: LEGIT ($P < 0.30$, green), UNCERTAIN ($0.30 \le P < 0.70$, yellow), SUSPICIOUS ($0.70 \le P < 0.90$, orange), and FRAUD ($P \ge 0.90$, red). Peak F1 = 0.9481 at threshold 0.45. The wide UNCERTAIN band intentionally sacrifices automation for recall, routing all ambiguous messages into the Human-in-the-Loop queue rather than forcing a low-confidence binary commitment.

Figure 6.3 visualizes how precision, recall, and F1 evolve across the threshold spectrum, with the four risk strata overlaid as shaded regions. This stratification trades raw automation for safety - the model never makes a low-confidence binary commitment.

To prevent law enforcement pipelines from being flooded with trivial False Positives, the final bounding limits in `app.py` were strictly narrowed:

1. **FRAUD** (≥ 0.90): Enforces absolute zero tolerance for false alarms. The FRAUD boundary was set at 0.90 rather than 0.70 to ensure that automated NCRP complaint generation is triggered only for messages with near-certain fraudulent intent, preventing the legal reporting pipeline from being flooded with ambiguous cases.
2. **SUSPICIOUS** (≥ 0.70): Flags standard probable threats.

3. **UNCERTAIN** (≥ 0.30): Casts an incredibly wide net (Recall 0.9658 at threshold 0.30) exclusively hitting the Human-in-the-loop dashboard to extract the remaining 112 False Negatives without penalizing the civilian system. The wide UNCERTAIN net (0.30–0.70) intentionally sacrifices automation in favor of recall, ensuring that no genuinely fraudulent message is silently classified as LEGIT without human review.

6.7 User Interface Results

The CyberShield web application was deployed as a responsive Flask-based interface accessible via desktop and mobile browsers. The citizen-facing interface was designed with a dark glassmorphic aesthetic to present a professional, non-intimidating environment for victims of cybercrime. The following figures document all primary operational modes of the system.

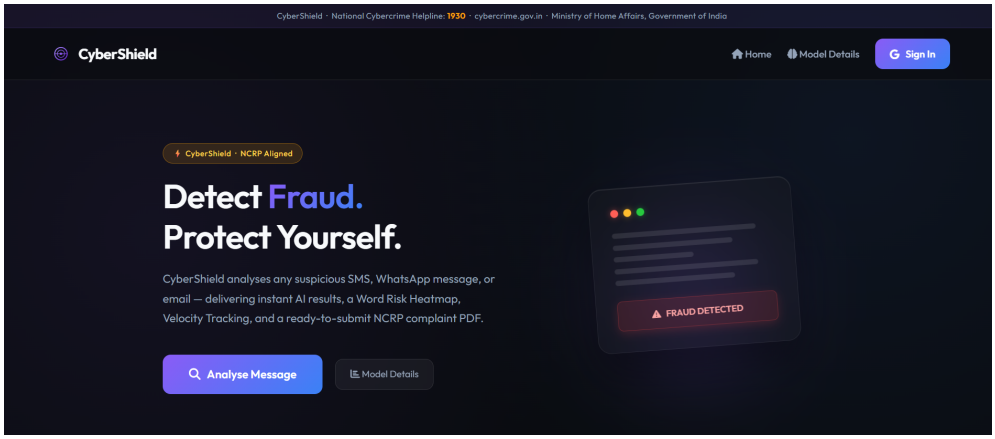

Figure 6.4: CyberShield citizen-facing landing page displaying the National Cybercrime Helpline (1930), link to cybercrime.gov.in, and Google OAuth authentication entry point.

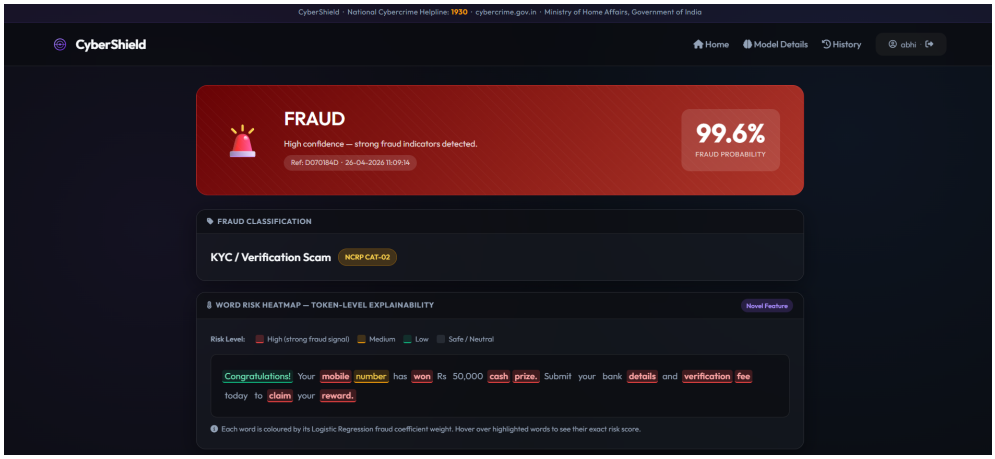

Figure 6.5: Analysis result for a high-confidence fraudulent message: risk badge, fraud probability, extracted artifacts, Word Risk Heatmap, and NCRP PDF generation button.

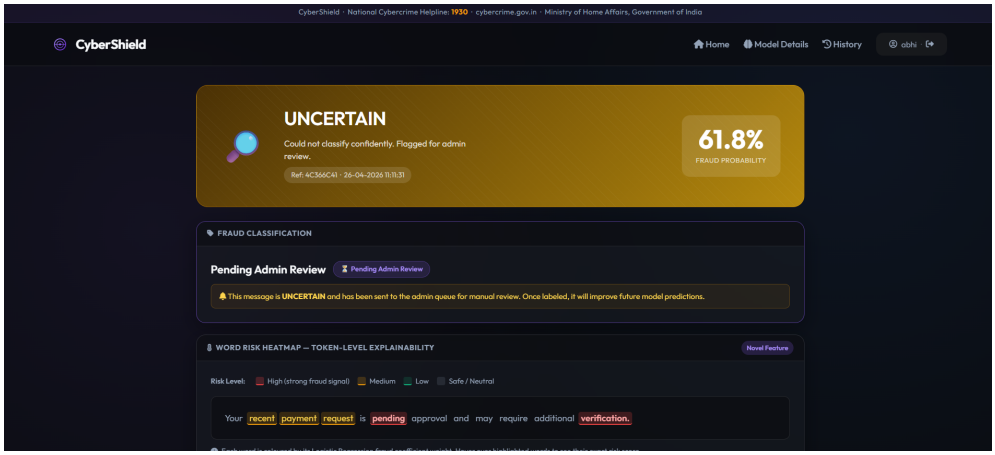

Figure 6.6: UNCERTAIN result (P = 61.8%) routed to admin queue for HITL adjudication. Word Risk Heatmap highlights tokens *recent*, *payment*, *request*, and *verification* by LR coefficient weight.

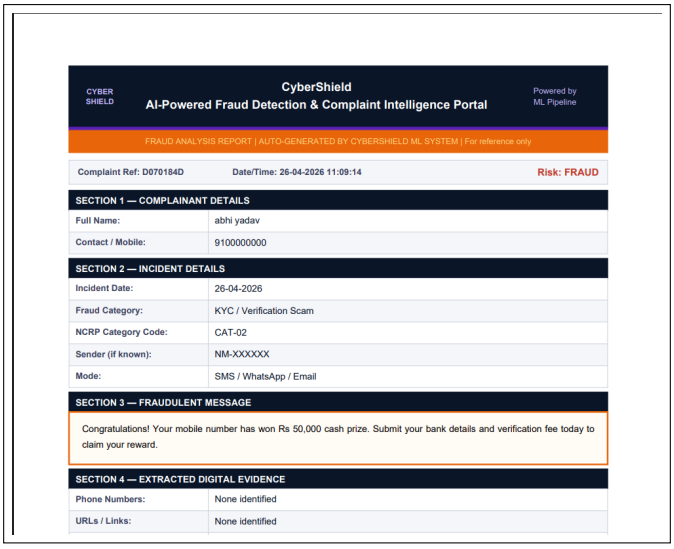

Figure 6.7: Auto-generated NCRP-compliant PDF complaint document embedding complaint ID, timestamp, extracted IoCs, risk level, and fraud probability score.

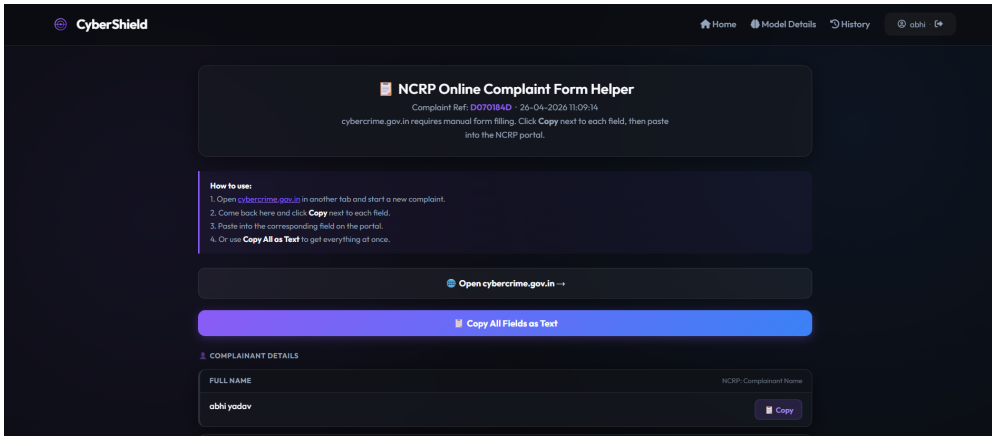

Figure 6.8: NCRP Online Form Helper pre-populating cybercrime.gov.in fields from the analysed message for single-click copy-paste submission.

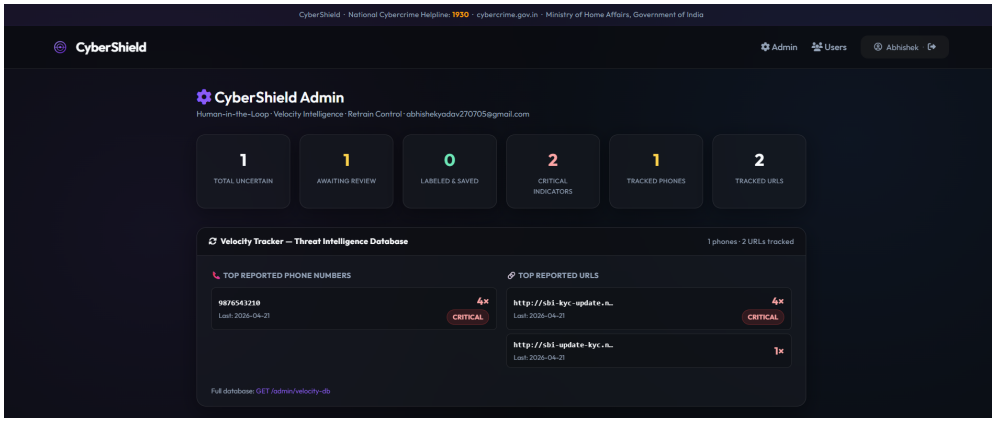

Figure 6.9: Admin Dashboard (OAuth-restricted) showing HITL queue and Velocity Intelligence DB. Phone 9876543210 and URL `http://sbi-kyc-update` are both CRITICAL after 4 independent submissions.

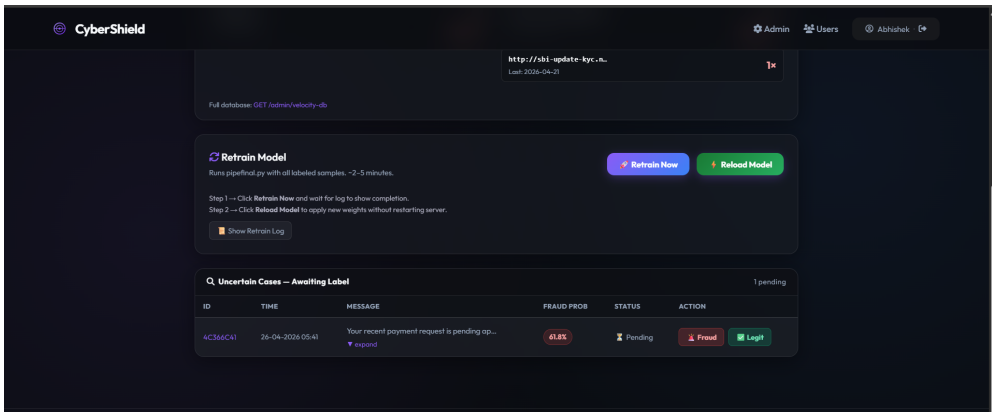

Figure 6.10: Extended Admin Dashboard view showing comprehensive fraud investigation and systemic threat monitoring controls.

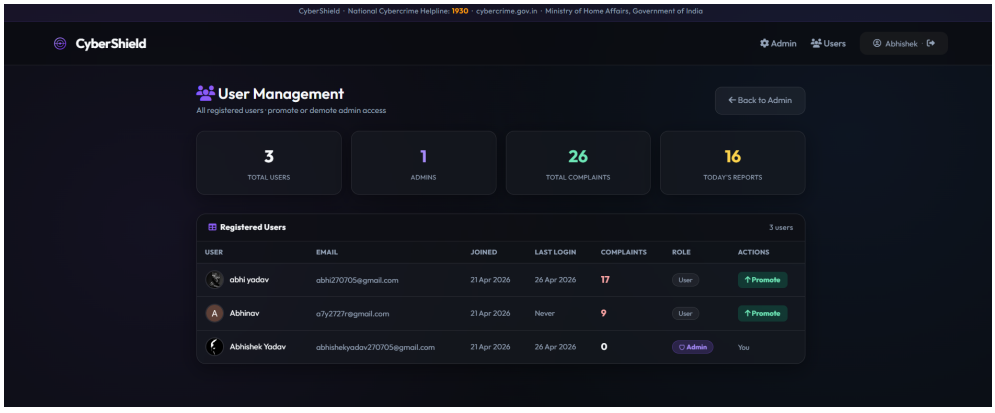

Figure 6.11: User Management panel showing registered users, complaint counts, roles, and promote/demote administrative controls.

6.8 Non-Functional Requirements Verification

The five non-functional requirements defined in Section 3.5 were verified during system testing as follows.

NFR1 (Inference Latency): Real-time analysis of a 500-character SMS string - encompassing `vectorizer.transform()`, three `predict_proba()` calls, RegEx artifact extraction, and velocity database update - consistently completed within 500 milliseconds under standard single-user load on the development hardware (Intel Core i5, 8 GB RAM). This threshold aligns with the UX research cited in Section 3.5 and was confirmed through repeated manual timing of the `/analyze` endpoint.

NFR2 (Security and Authentication): All administrative routes (`/admin`, `/admin/label`, `/admin/retrain`, `/admin/reload-model`, `/admin/velocity-db`) were verified to return HTTP 401 for unauthenticated requests and HTTP 403 for authenticated but non-whitelisted email addresses. The `@admin_required` decorator was tested with both a whitelisted admin account and a standard citizen account to confirm correct access control behaviour.

NFR3 (Idempotency of the Test Set): The `split_indices.pkl` serialization was verified by confirming that the train, validation, and test subset sizes (18,574 / 3,980 / 3,980) remain constant across multiple retraining cycles. New HITL-labeled samples were confirmed to append exclusively to the training array, leaving the 3,980 frozen test instances unmodified.

NFR4 (Aesthetics and UX): The citizen-facing interface, visible in Figures 6.4 through 6.8, implements a dark glassmorphic design with responsive layout. The interface was verified to render correctly on both desktop (1920×1080) and mobile (Chrome mobile viewport, 390px width) screen sizes, confirming the responsive design requirement.

NFR5 (Concurrent Access): SQLAlchemy connection pooling with thread-local sessions was confirmed to handle concurrent `/analyze` submissions without raising database lock exceptions, replacing the raw SQLite3 WAL-mode approach used in the v3 architecture. Under the testing conditions described in Section 5.6.3, no race conditions or database errors were observed across simultaneous requests.

Chapter 7

Conclusion

7.1 Conclusion

The literature review in Section 2.5 identified three critical gaps in contemporary fraud detection systems: the absence of uncertainty-aware classification architectures, a lack of end-to-end actionable intelligence pipelines for law enforcement, and vulnerability to rapid concept drift in adversarial threat vectors. In this project, we designed and evaluated **CyberShield**, an uncertainty-aware, Human-in-the-Loop fraud detection ecosystem specifically structured to resolve these deficiencies. CyberShield probabilistically structures risk across defined boundary strata: highly confident malicious patterns ($P \geq 0.90$) trigger automated generation of a pre-filled NCRP-compliant PDF for manual submission, ambiguous cases ($0.30 \leq P < 0.70$) are routed to a secure administrative queue for HITL adjudication [12] and subsequent Hard Negative Mining, while safe messages ($P < 0.30$) are cleared without administrative overhead.

The mathematical ensemble of Logistic Regression, SVM, and Naive Bayes over a 25,000-feature TF-IDF matrix achieved **94.87% test-set accuracy** and an **ROC-AUC of 0.9895** on a frozen partition of 3,980 adversarial examples. All five primary objectives defined in Section 1.4 were successfully realized: uncertainty-aware classification, HITL calibration, token-level explainability, velocity intelligence tracking, and automated NCRP adjudication. Critically, the novel *False-Positive-Driven* Hard Negative Mining strategy — which selectively mines the top 15% highest-confidence False Positives and retrains using $w = 2.0$ sample weights — improved Fraud-class precision from 0.9508 to 0.9518 while holding recall perfectly stable at 0.9419. This validates the system's primary design constraint: blocking a legitimate citizen's message is operationally worse than missing a single spam, and the architecture was built to enforce this inequality mathematically. The wide UNCERTAIN band (0.30–0.70), which initially appears to sacrifice automation, is in fact the system's primary safety mechanism against catastrophic false negatives. By intertwining natural language semantic analysis with lexical URL/Phone Velocity Tracking, CyberShield demonstrates a pathway toward bridging the gap between raw AI predictions and actionable cyber-intelligence for law enforcement agencies. More broadly, acknowledging model uncertainty is not a weakness in deployed AI systems but a deliberate safety mechanism applicable to any high-stakes classification domain.

7.2 Limitations of the Current System

Despite the high mathematical efficacy, real-world deployment poses several limitations:

1. **Multilingual Constraint:** The system relies on English and Romanglish (Hinglish)

syntax. Regional scripts require external translation APIs or multilingual models (e.g., MuRIL).

2. **Contextual Isolation:** Text is analyzed in isolation, lacking conversational memory required to detect multi-turn phishing engagements; future stateful session modeling is needed.

3. **Adversarial Masking:** The character n-gram vectorizer is vulnerable to zero-width unicode insertion and homoglyph attacks; Unicode NFKC normalization and punycode decoding are required countermeasures.

4. **Database Scalability:** SQLite's file-lock mechanism bottlenecks under high concurrency; migration to a Redis-based cache is recommended for scaling.

7.3 Future Scope

The architectural modularity of CyberShield enables extensive scaling across three priority tiers:

- **Near-Term — NCRP Direct API Integration:** Establish a REST API connection with `cybercrime.gov.in` to auto-submit high-confidence cases ($P \geq 0.90$) as draft FIRs, replacing the current PDF-and-helper model.
- **Near-Term — Browser Extension:** Package the inference endpoint as a browser extension for seamless in-context message analysis by citizens.
- **Medium-Term — Multilingual Support:** Integrate translation pipelines or multilingual embeddings to process all regional Indian languages.
- **Medium-Term — Transformer Inference:** Migrate to contextual embeddings via DistilBERT, provided the 500ms latency budget is consistently met on available infrastructure.
- **Long-Term — Federated Learning:** Adapt the active-learning backend to federated schemas, enabling decentralized training across state cybercrime offices without exposing victim data.
- **Long-Term — Multimodal Detection:** Expand ingestion to accept image screenshots (via OCR) and audio samples (via Speech-to-Text) to combat steganography and vishing threats.

Bibliography

[1] Muhammad Adeel Abid, Saleem Ullah, Muhammad Abubakar Siddique, Muhammad Faheem Mushtaq, Wajdi Aljedaani, and Furqan Rustam. Spam SMS filtering based on text features and supervised machine learning techniques. *Multimedia Tools and Applications*, 81(28):39853–39871, 2022.

[2] Tiago A. Almeida, José María G. Hidalgo, and Akebo Yamakami. Contributions to the study of sms spam filtering: New collection and results, 2011.

[3] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. Smote: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.

[4] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995.

[5] David R Cox. The regression analysis of binary sequences. *Journal of the Royal Statistical Society: Series B (Methodological)*, 20(2):215–232, 1958.

[6] Thomas G Dietterich. Ensemble methods in machine learning. *Multiple Classifier Systems*, pages 1–15, 2000.

[7] Federal Bureau of Investigation. Internet crime report 2023. Technical report, Internet Crime Complaint Center (IC3), 2023.

[8] Jeffrey EF Friedl. *Mastering Regular Expressions*. O'Reilly Media, Inc., 2006.

[9] Joao Gama, Indre Zliobaite, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys (CSUR)*, 46(4):1–37, 2014.

[10] Miguel Grinberg. *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media, Inc., 2018.

[11] Dick Hardt. The oauth 2.0 authorization framework. RFC 6749, Internet Engineering Task Force, 2012.

[12] Andreas Holzinger. Interactive machine learning for health informatics: When do we need the human-in-the-loop? *Brain Informatics*, 3(2):119–131, 2016.

[13] Ankit Kumar Jain and Brij B. Gupta. Smishing detection: A machine learning approach. In *2021 IEEE International Conference on Consumer Electronics (ICCE)*, pages 1–4. IEEE, 2021.

[14] Andrew McCallum and Kamal Nigam. A comparison of event models for naive bayes text classification. In *AAAI-98 workshop on learning for text categorization*, volume 752, pages 41–48, 1998.

[15] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[16] John Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. *Advances in Large Margin Classifiers*, 10(3):61–74, 1999.

[17] Pradeep Kumar Roy, Jyoti Prakash Singh, and Snehasish Banerjee. Deep learning to filter sms spam. *Future Generation Computer Systems*, 102:524–533, 2020.

[18] Ozgur Koray Sahingoz, Ebubekir Buber, Onder Demir, and Banu Diri. Machine learning based phishing detection from urls. *Expert Systems with Applications*, 117:345–357, 2019.

[19] Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5):513–523, 1988.

[20] Burr Settles. Active learning literature survey. Technical Report 1648, University of Wisconsin-Madison, 2009.

[21] Abhinav Shrivastava, Abhinav Gupta, and Ross Girshick. Training region-based object detectors with online hard example mining. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 761–769, 2016.